

UnifAI / Raksha Enterprise AI Platform

Complete Documentation Suite & Master Directory Index

Classification: Enterprise System Documentation

Suite Release: Version 2.4.0 (Baseline commit **16d4c3e**)

Storage Directory: **/pdf/** & **/docs/pdf/**

Generated Date: October 2026

1. Documentation Suite Overview

This documentation folder (**pdf/**) contains the complete technical, operational, architectural, and security manuals for the **UnifAI Enterprise AI Gateway** and **Raksha Browser Guard** endpoint protection suite.

All documentation is provided in both **Markdown (.md)** format for source tracking and **Adobe PDF (.pdf)** format for distribution, printing, and archiving.

2. Document Catalog & Coverage

#	Document File	Formats Available	Primary Audience	Description & Coverage
1	Technical Documentation	01_TECHNICAL_DOCUMENTATION.md 01_Technical_Documentation.pdf	Architects, Lead Developers, Security Engineers	Complete system overview, dual-plane architecture, Go 1.26+ fasthttp gateway engine, core/providers (~28 AI vendors), universal schemas, MCP client, key selectors, framework/configstore , framework/logstore , RBAC (33 resources × 6 operations, section grants), plugin pipeline, HTTP routes, ConfigStore & Browser AI tables, Browser Guard agent (apps/browser-guard/agent), mitmproxy addon parts, React 19 UI architecture, security model, and developer test workflows.
2	Server Implementation Guide	02_SERVER_IMPLEMENTATION_GUIDE.md 02_Server_Implementation_Guide.pdf	DevOps, SRE, Systems Administrators	Deployment specifications, system requirements (CPU, RAM, Disk, Kernel sysctl limits), complete .env configuration breakdown, config.json reference, multi-stage Docker build, volumes, network proxy profile, PostgreSQL database setup, Nginx reverse proxy configuration (SSL, HTTP/2, WebSocket upgrades, unbuffered SSE streaming), Kubernetes Helm chart (deploy/helm/unifai), operations (start/stop/restart, logs, health, backups, rolling upgrades, HA), and troubleshooting matrix.

#	Document File	Formats Available	Primary Audience	Description & Coverage
3	Admin Console Guide	03_ADMIN_CONSOLE_GUIDE.md 03_Admin_Console_Guide.pdf	Platform Admins, InfoSec, Workspace Managers	Initial onboarding checklist, roles (admin , sub_admin , user , custom roles), User lifecycle management (approvals, rejections, SCIM), Access Profiles, Virtual Keys (budgets, RPM/TPM rate limits), Model Providers and catalog, Complexity Router, routing rules, circuit breakers, Guardrails vs Guard Rules, Browser AI control plane (Target Websites, Guard Rules, interaction controls, Fleet configuration, Guard Agents, Setup & packaging, company uninstall keys), and Workspace Config views.
4	End-User & Developer Guide	04_USER_GUIDE.md 04_User_Guide.pdf	Employees, Developers, Data Scientists	Platform introduction in plain language, account registration, email verification, login, password recovery, navigating the Workspace UI (Prompt Repository, Skills Repository, Model Playground, Personal Observability), Developer API integration (Virtual Keys, curl , Python OpenAI SDK, TypeScript, LangChain, Cursor IDE), Desktop Browser Guard employee guide (Windows & macOS installation, in-browser DLP alerts: Block, Redact, Warn, and privacy protections).
5	Database Configuration Guide	05_DATABASE_CONFIGURATION_GUIDE.md 05_Database_Configuration_Guide.pdf	DBAs, DevOps, Infrastructure Engineers	PostgreSQL 16 installation on Ubuntu/Debian/RHEL/Docker, UTF-8 schemas and user provisioning SQL (agent_unify , raksha_new), pg_hba.conf network security (SCRAM-SHA-256), SSL/TLS database encryption (generating server.key & server.crt , DB_SSL_MODE=require), GORM table reference, PgBouncer connection pooling setup (pgbouncer.ini), production performance tuning parameters for postgresql.conf , automated daily backup bash script with 30-day retention and crontab scheduling, and disaster recovery restore procedure.
6	Docker & SSL/TLS Configuration Guide	06_DOCKER_AND_SSL_CONFIGURATION_GUIDE.md 06_Docker_and_SSL_Configuration_Guide.pdf	DevOps, Cloud Engineers, SREs, SecOps	Multi-container Docker Compose architecture, exhaustive Docker command reference (up , down , logs , exec , build , prune), Public HTTPS with Let's Encrypt / Certbot (standalone, webroot, nginx plugin, auto-renew crontab), OpenSSL self-signed certificate generation (4096-bit RSA, SAN extensions), Browser Guard Root CA generation (raksha-ca.crt , raksha-ca.key , mitmproxy-ca.pem), client trust store installation (Windows certutil , macOS security , Linux update-ca-certificates , Firefox NSS), Nginx reverse proxy configuration, and SSL handshake diagnostic commands.

3. Quick Reference: Critical System Ports

- **Public Web / HTTPS: 443** (Nginx / Caddy Reverse Proxy)
- **Plain HTTP Redirect: 80** (Certbot ACME challenge & HTTPS redirect)
- **UnifAI Go Backend Core: 6000** (Internal localhost bind: **127.0.0.1:6000**)
- **Docker Network Proxy: 18182** (Optional compose profile **network-proxy**)
- **PostgreSQL Database: 5432** (or **6432** with PgBouncer)
- **Local Ollama LLM Service: 11434** (Internal service for AI Guard Bot)
- **Browser Guard Local MitM Proxy: 18103** (Bound to client laptop loopback **127.0.0.1**)
- **Browser Guard Local PAC Server: 18195** (Bound to client laptop loopback **127.0.0.1**)

UnifAI Documentation Suite - Built for Enterprise AI Governance & Security.

UnifAI - Technical Documentation

Audience: developers, architects, integrators

Scope: architecture, modules, data model, HTTP surface, plugins, Browser AI internals, UI, security model

Baseline: repo commit **16d4c3e**

1. System overview

```

##### UnifAI Go binary (fasthttp) #####
/v1/* inference (OpenAI-compatible) ■ AI apps #####
/openai|anthropic|genai|bedrock|cohere|cursor|litellm|langchain|pydanticai pass-through ■ /api/* dashboard,
governance, observability, Browser AI control plane ■ Admin UI ##### ■ /ws, /v1/realtime WebSocket +
realtime transports ■ (embedded) ■ /health, /robots.txt, /metrics, /pprof (dev) ■ #####
##### ConfigStore
(Postgres) LogsStore (Postgres) users, roles, VKs, keys, LLM logs, MCP logs, Browser AI logs, providers,
settings search logs, dashboard aggregates ■ ##### ■ UnifAI core (Go) ■ →
provider SDKs/HTTP → ~28 LLM vendors ■ plugins, MCP client ■ ##### Guard fleet
(endpoint agents + network proxy) ■ HTTPS ■ /api/browser-ai/* ■ Postgres browser_ai *

```

Two data planes exist in the same binary:

1. **Data plane** (`/v1/*`, pass-through integrations) - request/response proxying to AI providers, with governance, guardrails, caching, logging, tracing.
2. **Control plane** (`/api/*`) - the dashboard: configuration, RBAC, observability and the whole Browser AI guardrail control plane.

2. Technology stack

Layer	Technology	Source of truth
Gateway	Go 1.26.4, valyala/fasthttp , fasthttp/router , automaxprocs	go.work , transports/unifai-http/main.go
Persistence	GORM; Postgres / MySQL / SQLite (CGO go-sqlite3 , sqlite_static)	framework/configstore , framework/logstore
Provider SDKs	Vendor HTTP clients + official SDKs	core/providers/*
Plugins	Go interfaces, optional .so dynamic loading (SharedObjectPluginLoader)	plugins/ , server/plugins.go
MCP	Model Context Protocol client/server, OAuth2 sessions	core/mcp , handlers/mcp*.go
Frontend	React 19, TypeScript 5.9, Vite 8, TanStack Router/Table, Redux Toolkit, Radix UI, Tailwind 4, Monaco, Recharts, Zod	ui/package.json
Guard agent	Python 3.11+, mitmproxy addon, PyInstaller, Inno Setup (Windows), pkg/dmg scripting (macOS)	apps/browser-guard/**
Deploy	Docker (multi-stage), docker compose, Helm (Postgres, Redis, Qdrant, Weaviate, HPA, Ingress)	deploy/** , docker-compose.yml
Tooling	Make targets, gotestsum, Playwright, newman, oxlint/oxfmt	Makefile

3. Repository layout

core/ Go module: the AI engine providers/<vendor>/ one package per provider (chat, responses, images, audio, video, batch...) schemas/ shared types (requests, responses, plugin interfaces, errors) mcp/ MCP client, code mode, credential store keyselectors/ weighted/random key selection strategies network/ HTTP client utilities (retries, timeouts, proxies) framework/ Shared infrastructure configstore/ ConfigStore + WorkspaceStore; tables/ has every GORM model logstore/ LLM/MCP/Browser AI log storage, aggregations, matviews, cleaners rbac/ Resource×Operation enforcement, route → requirement mapping encrypt/ AES encryption at rest for secrets in DB cluster/, loadbalancer/, routing/, circuitbreaker/, kvstore/, featureflags/ connectors/, objectstore/, vectorstore/, oauth2/, mailer/, alerts/, tracing/ migrator/, mysqlconn/, postgresconn/ DB drivers + schema migration helpers plugins/ telemetry, prompts, logging, governance, otel, semanticcache, compat, maxim, guardrails, connectors, modelcatalogresolver, mocker, jsonparser transports/unifai-http/ The server binary main.go flags, logger, version banner, bootstrap/start server/server.go UnifAIHTTPServer: bootstrap, middleware assembly, graceful shutdown server/plugins.go built-in plugin instantiation + ordering handlers/*.go every /api and /v1 handler (one file per feature area) integrations/*.go pass-through provider surfaces websocket/, ui/ WS pool/sessions; embedded UI assets ui/ React dashboard (Vite). pages under ui/app/workspace/** apps/browser-guard/ Python Guard agent + mitmproxy addon + installers + release artifacts configs/ config.json, mcp-library.json, model-parameters.json, pricing.json deploy/ docker/, helm/unifai/ docs/ this documentation set

4. Core module (core/)

4.1 Providers

Each directory under **core/providers/<vendor>** implements the shared provider contract from **core/schemas**. A provider package contains files such as **chat.go**, **responses.go**, **images.go**, **speech.go**, **transcription.go**, **videos.go**, **batch.go**, **models.go**, **errors.go**, **types.go**, **utils.go**. Capability differences that matter (from the files present):

Provider(s)	Notable capability files
openai	chat, responses, responseslifecycle, text, images, speech, transcription, videos, batch, files, embedding, realtime, websocket, large_payload, passthrough_usage
anthropic, gemini	chat, responses, count_tokens / models, embedding (gemini), types
vertex	chat, batch, cachedcontents, count_tokens, embedding, rerank, models
bedrock, bedrockmantle	chat, responses, embedding, images, batch
replicate, runware, runway	chat, images, videos, files
mistral	adds ocr.go on top of chat/transcription/cachedcontents
elevenlabs	audio-only
ollama, sgl, vllm, openrouter, parasail, opencode, groq, cerebras, deepseek, fireworks, huggingface, nebius, perplexity, xai, cohere	vendor subsets (mostly OpenAI-compatible shapes)
openaicompat	generic OpenAI-compatible provider + registry.go for many compatible endpoints
azure	Azure OpenAI, deployment-aware

Shared helpers live in **core/providers/utils**: **decompression.go**, **sse.go**, **stream.go**, **streamterminaldetector.go**, **pagination.go**, **images.go**, **audio.go**, **file.go**, **videos.go**,

bodysigner.go (request body signing for AWS-style auth), **modelparamscache.go**, **largerresponse.go**, **passthrough_stream.go**, **fetch.go**, **models.go**, **utils.go**.

4.2 Schemas

core/schemas holds the types shared by the gateway and every plugin: chat / responses / text / embedding / image / audio / video request-response types, **UnifAIConfig**, **UnifAIContext**, **ProviderConfig**, **NetworkConfig**, the plugin interfaces (**BasePlugin**, **LLMPlugin**, **MCPPlugin**, **ObservabilityPlugin**, **HTTPTransportPlugin**, **UnifAIHTTPMiddleware**), error types and log level/output enums.

4.3 MCP client

core/mcp implements the Model Context Protocol client used by the MCP Gateway: connection lifecycle, tool discovery, tool execution, per-user credentials (**credstore**), "code mode" (**codemode**) and utilities. The HTTP layer wraps it in **handlers/mcp.go** (clients, library, tool groups), **handlers/mcpserver.go**, **handlers/mcpsessions.go** and the OAuth2 set (**mcpoauth2.go**, **mcpoauth2discovery.go**, **mcpoauth2issuance.go**, **mcpoauth2consent.go**, **mcpoauth2sessions.go**, **mcpoauth2jwt.go**).

4.4 Key selection and networking

- **core/keyselectors** - weighted/random selection across multiple keys per provider.
- **core/network** - the outbound HTTP client for provider calls (proxy, TLS, timeouts, retries), driven by each provider's **NetworkConfig**.

5. Framework module (**framework/**)

5.1 ConfigStore

framework/configstore is the configuration database layer.

- Drivers: **postgres.go**, **mysql.go**, **sqlite.go**, **rdb.go**; interface in **store.go**; errors in **errors.go**.
- **migrations.go** plus **framework/migrator** (**addcolumn.go**, **dropcolumn.go**) handle schema evolution.
- **tables/** holds one file per domain: **user.go**, **workspace.go**, **enterprise.go**, **team.go**, **customer.go**, **virtualkey.go**, **key.go**, **provider.go**, **model.go**, **modelconfig.go**, **modelpricing.go**, **modelparameters.go**, **pricingoverride.go**, **budget.go**, **ratelimit.go**, **routingrules.go**, **mcp*.go**, **mcpoauth2*.go**, **prompts.go**, **promptVersions.go**, **promptSessions.go**, **skills.go**, **folders.go**, **featureflag.go**, **plugin.go**, **config.go**, **clientconfig.go**, **env.go**, **sessions.go**, **temptokens.go**, **dlock.go** (distributed locks), **logstore.go**, **vectorstore.go**, **smtp.go**, **confighash.go**.
 - **workspace.go** seeds the **RBAC catalog**, the three **system roles**, and exposes the resource/operation/name catalogs used by backend and UI.

- Secrets in the DB are encrypted with **framework/encrypt** when **UNIFAI_ENCRYPTION_KEY** is set (**tables/encryption.go**, **configstore/encryption.go**).

5.2 LogsStore

framework/logstore stores and aggregates request logs.

- Files: **logger.go**, **store.go**, **rdb.go**, **postgres.go**, **mysql.go**, **sqlite.go**, **hybrid.go**.
- **browser_ai.go** is the **entire Browser AI persistence layer** (see §9).
- Aggregations for dashboard/log pages: **matviews.go** (materialized views), **tables.go**, **cleaner.go** (retention), **asyncjob.go** (async inference jobs), **payload.go**.

5.3 RBAC (two enforcement layers, both applied to **/api/***)

1. **Resource × Operation** - **framework/rbac/enforce.go**, **PathRequirementFor(method, path)**

returns the requirement for a route (or **nil** = not RBAC-gated). **admin** resolves to *allow all*;

an unknown/unseeded role yields an **empty set = deny**. **View** and **Read** are interchangeable.

2. **Sidebar section grants** - **framework/rbac/sections.go**, **SectionRequirementFor(method, path)**,

used for section-scoped sessions. Keys are **parent** or **parent/child**; **parent/*** accepts the

parent or any child. Legacy **observability/browser-ai** is normalised to **browser-ai**.

Resource catalog (33): `GuardrailsConfig, GuardrailsProviders, GuardrailRules, UserProvisioning,

Cluster, Settings, Users, Logs, Observability, Dashboard, VirtualKeys, ModelProvider, Plugins,

MCPGateway, MCPToolGroups, MCPLogs, AdaptiveRouter, AuditLogs, Customers, Teams, RBAC, Governance,

RoutingRules, PromptRepository, PromptDeploymentStrategy, SkillsRepository, AccessProfiles, APIKeys,

Inference, Metrics, FeatureFlags, CircuitBreaker`

Operation catalog (6): **Read, View, Create, Update, Delete, Download**

System roles (auto-seeded in **configstore/workspace.go**, then merged on every boot so new

permissions are picked up - except for roles a customer has customised):

Role	DAC	Grants
admin	all-data	every permission
sub_admin	all-data	Read/View on Dashboard, Inference, Observability, MCPLogs, RoutingRules, Metrics ; full (Read/View/Create/Update/Delete/Download) on Logs - this is what gates all Browser AI write APIs; full on VirtualKeys, PromptRepository ; View/Read/Update on Governance ; View/Read/Create/Update on MCPGateway, MCPToolGroups ; View/Read on ModelProvider . No master provider keys, no system settings
user	own-data	Read/View only , on Dashboard, Logs, Inference, PromptRepository, Observability, MCPGateway (the readIDs set). The UI labels this "Prompt Repository only"; the backend also allows read-only views of logs/observability/MCP. Use allowed_sections to narrow what a user actually sees

Custom roles are created in the UI. **admin** is always re-synced to the full catalog.

*Browser AI note: every **/api/browser-ai/*** route maps to the **Logs** RBAC resource*

*(**PathRequirementFor**), so a role needs **Logs: View/Update** (or the **browser-ai** section grant)*

to see the Browser AI pages.

5.4 Other framework packages

Package	Purpose	API surface
cluster	multi-node config propagation	/api/cluster , /internal/cluster/kv
loadbalancer	adaptive routing across providers/keys	/api/load-balancer , /api/load-balancer/routes
routing	routing-rule engine	/api/governance/routing-rules , /api/routing-rules
circuitbreaker	failure isolation policies	/api/circuit-breaker/policies , /state
featureflags	server-side flags	/api/feature-flags
rbac, oauth2, temptoken	permissions, OAuth2 provider, short-lived tokens	/api/rbac/* , /api/scim/oauth/*
mailer, alerts	SMTP and alert delivery (webhooks)	/api/smtp-config , /api/alert-channels
connectors	external sinks (BigQuery, Kafka, Datadog, New Relic, PubSub)	/api/connectors
objectstore, vectorstore, kvstore	blob / vector / KV backends (Qdrant, Weaviate, Redis)	/api/vector-store-config , /api/cache
tracing	trace store + spans	/api/logs trace views
querscope, mcpcatalog, mcptoolgroups, mcp_headers, modelcatalog, encrypt, envutils	supporting services	-
streaming	SSE/stream utilities shared by transports	-

6. Plugins (plugins/)

A plugin implements one or more core interfaces and is loaded either as a **built-in** (by name) or as a **dynamic .so** via **SharedObjectPluginLoader (server/plugins.go)**.

Built-in load order (fixed placement indexes):

#	Plugin	Role
1	telemetry	request metrics/counters
2	prompts	prompt repository integration
3	logging	LLM/MCP log persistence
4	governance	virtual keys, budgets, rate limits, teams, customers, business units
5	otel	OpenTelemetry export
6	semanticcache	embedding-based response caching
7	compat	parameter compatibility/conversion between client and provider shapes

#	Plugin	Role
8	maxim	Maxim observability integration
-	guardrails	rule-based input/output validation + provider config
-	connectors	external log/analytics sinks
-	modelcatalogresolver	resolves model metadata/pricing
-	mock	mock provider for tests/E2E
-	jsonparser	tolerant JSON parsing helpers

Plugin config lives in **config.json** under **plugins**; plugins can be enabled/disabled and reordered at runtime (**GET/POST /api/plugins**). A plugin that fails to load is marked disabled and reported in the startup plugin-status table (**plugin status: <name> - <status>**).

7. HTTP surface (**transports/unifai-http**)

7.1 Entry point and flags

main.go:

Flag	Env fallback	Default
-port	APP_PORT	8001
-host	APP_HOST / UNIFAI_HOST	localhost
-app-dir	APP_DIR	./data
-log-level	LOG_LEVEL	info
-log-style	LOG_STYLE	json

Startup prints an ASCII banner (**-ldflags -X main.Version=...**), starts the pprof server, configures the logger, then **Bootstrap()** -> **Start()**.

7.2 Middleware assembly

Server handler wrapper (**server.go**):

```
SecurityHeadersMiddleware → CORS → RequestDecompression → router
```

API middleware chain (**apiMiddlewares**, applied to every **/api/*** route):

```
AuthMiddleware.APIMiddleware() session cookie / Bearer session token WorkspaceAuditMiddleware audits  
mutating /api/* into Audit Logs RBACMiddleware resource × operation + section grants
```

Inference middleware chain (**/v1/*** and pass-through):

```
TracingMiddleware → TransportInterceptor → AuthMiddleware.InferenceMiddleware → handler
```

TransportInterceptor runs **HTTPTransportPostHook** plugins and captures their log entries; the tracing defer runs afterwards so plugin logs land inside the trace.

7.3 Request identification on inference

- **Authorization: Bearer <virtual-key|provider-key>** or **x-api-key: <key>**.
- Dashboard-originated inference is enriched from the session cookie

(**enrichInferenceFromDashboardSession**); provider headers are normalised to a single

Authorization: Bearer ... (**x-api-key** is deleted).

- Password-as-Bearer (base64 **user:pass**) is **removed** - only session tokens are accepted as Bearer.
- Virtual Key self-service quota: **GET /api/governance/virtual-keys/quota** (the VK header is the credential).

7.4 Route groups

Group	Representative routes	Handler file(s)
Health / UI	GET /health , GET /robots.txt , / (SPA)	health.go , ui.go
Session / auth	/api/session/login , /logout , /register , /register/verify , /register/resend , /forgot-password , /verify-otp , /reset-password , /forgot-username , /is-auth-enabled , /ws-ticket , /api/session/users*	session.go , session_register_verify.go , auth_security.go
Workspace / RBAC	/api/users/{id}/role , /api/roles* , /api/permissions , /api/resources , /api/operations , /api/rbac/* , /api/access-profiles*	workspace.go , rbac.go , rbac_enforce.go , accessprofiles.go
Governance	/api/governance/virtual-keys* , /customers* , /teams* , /business-units* , /routing-rules* , /complexity-analyzer-config , /api/guardrails/*	governance.go , businessunits.go , team_members.go , guardrails.go
Inference	/v1/chat/completions , /v1/completions , /v1/responses* , /v1/embeddings , /v1/rerank , /v1/ocr , /v1/audio/* , /v1/images/* , /v1/videos* , /v1/batches* , /v1/files* , /v1/containers* , /v1/realtime , /v1/async*	inference.go , asynclineference.go , realtime_*.go , webrtc_realtime.go
Pass-through	/openai/* , /anthropic/* , /genai/* , /bedrock/* , /cohere/* , /cursor/* , /litellm/* , /langchain/* , /pydanticai/*	integrations/*.go
Observability	/api/logs* , /api/logs/dashboard , /api/logs/filterdata , /api/logs/rankings* , /api/logs/histogram/* , /api/mcp-logs* , /api/audit-logs , /api/connectors , /api/alert-channels	logging.go , auditlogs.go , alertchannels.go , integrations.go
Providers / models	/api/providers* , /api/keys* , /api/models* , /api/model-configs*	providers.go , provider_keys.go , list_models_vk.go
MCP	/api/mcp/clients , /api/mcp/library* , /api/mcp/client* , /api/mcp/tool-groups* , /api/mcp-logs , /api/mcp/inference	mcp.go , mcpserver.go , mcpsessions.go , mcptoolgroups.go , mcpheaders.go
Prompts / skills	/api/prompt-repo* , /api/prompt-deployments , /api/skills* , /api/skills/serve/*	prompts.go , prompts_lifecycle.go , skills.go , skills_serving.go
Config / platform	/api/config , /api/proxy-config , /api/vector-store-config , /api/smtp-config , /api/cache , /api/plugins , /api/feature-flags , /api/cluster , /api/circuit-breaker/* , /api/load-balancer*	config.go , plugins.go , featureflags.go , cluster.go , circuitbreaker.go , cache.go
Enterprise / SCIM	/api/scim/* , /api/scim/oauth/*	scim.go , scim_handlers.go , scim_oauth.go , scim_auth.go

Group	Representative routes	Handler file(s)
Browser AI	see §9	browser_ai*.go
WebSocket	/ws , WS tickets, responses/realtime WS	websocket.go, wsrealtime.go, wsresponses.go, ws_ticket.go
Dev	/api/dev/* + pprof (dev mode only)	devpprof*.go

7.5 Health and observability endpoints

- **GET /health** - pings ConfigStore, LogsStore and VectorStore concurrently (10 s timeout) and returns **{"status": "ok", "components": {"db_pings": "ok"}}**; returns HTTP 503 with the first failing component otherwise. Can be short-circuited by **client.disable_db_pings_in_health**.
- **GET /robots.txt** - static **Allow: /**.
- **GET /api/version** - build version.
- Logging skip list (no request logging): **/health, /_next, /api/dev**.
- Metrics (Prometheus) and traces come from the telemetry/otel plugins and the Logs API.
- Dev builds additionally expose pprof handlers (gated by **devpprof_prod.go** in release builds).

8. Data model (key tables)

8.1 ConfigStore tables

- **governance_users** - `id, username(uniq), email, password, role(admin|sub\_admin|user), status (pending|approved|rejected|email\_unverified|disabled), budget, rate\_limit, budget\_id, rate\_limit\_id, allowed\_prompt\_repos, allowed\_sections, reviewed\_at, external\_id, created\_at, updated\_at`.
- **IsApproved()** treats an empty status as approved (legacy rows).
- **rbac_roles** - `id, name(uniq), description, is\_system\_role, dac(all-data|own-data), permission\_ids(JSON), created\_at, updated\_at`.
- **access_profiles** - `id, name(uniq), description, is\_active, version, calendar\_aligned, tags(JSON), spec(JSON), created\_at, updated\_at` (reusable governance policy templates with activate / deactivate / clone operations).
- **governance_business_units** - **id, name(uniq), team_ids(JSON), budget(JSON), rate_limit(JSON)**.
- Grouped in **tables/workspace.go**: **alert_channels, circuit_breaker_policies, mcp_tool_groups, prompt_deployments, workspace_settings, virtual_key_users, virtual_key_teams, virtual_key_customers, audit_logs**.
- Plus the core governance tables: teams, customers, virtual keys, provider keys, models, model configs, pricing + pricing overrides, budgets, rate limits, routing rules, prompts/versions/sessions, skills, folders, feature flags, plugins, sessions, temp tokens, distributed locks, SMTP, vector-store config, MCP clients/library/OAuth2 issuance.

8.2 Browser AI tables (**framework/logstore/browser_ai.go**)

Table	Purpose	Notable columns
browser_ai_logs	one row per intercepted prompt / upload	id, timestamp, platform, domain(host), user_prompt_full, action, status, rule_triggered, risk_score, predictive_risk, agent_id, agent_hostname, client_ip, reply_bot_text, attachment_name/type/size/path/expiry, metadata(JSON)
browser_ai_search_logs	search-engine queries + clicked results	id, timestamp, engine, browser, is_incognito, query, clicked_url, clicked_title, url, host, agent_id, predictive_risk
browser_guard_rules	guard rules	id, name, pattern, rule_type(regex or ai_bot), severity, action, warning_message, active, description, bot_provider, bot_model, bot_prompt, bot_reference_image, bot_reference_image_type
browser_ai_target_websites	monitored AI sites	id, domain(uniq), platform_name, monitored, block_entire_website, host_role(ui/chat/file), status(MONITORED/PAUSED/BLOCKED), parent_id
browser_ai_controls_settings	single-row interaction policy (browser-controls-default)	upload/download warning text and policy toggles
browser_ai_agents	Guard fleet inventory	id, hostname, agent_type(endpoint/network), status(active/uninstall_pending/uninstalled), last_seen_at, health_detail, contact_email(+pinned), uninstall_key_hash, uninstall_key_enc, has_uninstall_key, uninstall_key_rotated_at, proxy_bundle_sha
browser_ai_agent_settings	company-level uninstall key	id, uninstall_key_enc
browser_guard_fleet_config	company Guard defaults that agents pull on heartbeat	id, ... notes
browser_guard_rebuild_logs	Guard package rebuild history	id, status(success/failed), bundle_sha, message, log, created_at
browser_ai_warning_emails	employee warning e-mail log	id, sent_at, to_email, message, agent_id, error_detail

- Attachment bytes stay on disk (**APP_DIR/attachments**); the DB stores size, path and expiry metadata.
- **BrowserAIManager** (same file) implements every CRUD/aggregation API used by the handlers,

including **GuardSeverityScore** (severity -> predictive risk score + label) and

NormalizeBrowserAIAgentType.

9. Browser AI (UnifAI Guard) - technical detail

9.1 Backend routes (**handlers/browser_ai.go** registers all of these)

Logs / insights

```
GET /api/browser-ai/logs list + filter prompt logs GET /api/browser-ai/logs/stats aggregated stats DELETE
/api/browser-ai/logs delete all POST /api/browser-ai/logs/bulk-delete DELETE /api/browser-ai/logs/{id} GET
/api/browser-ai/search-logs POST /api/browser-ai/search-logs agent ingest DELETE /api/browser-ai/search-logs
POST /api/browser-ai/search-logs/bulk-delete DELETE /api/browser-ai/search-logs/{id} GET
/api/browser-ai/insights/stats Guard Insights
```

Rules / controls / targets

```
GET|POST /api/browser-ai/rules + POST /rules/import, PUT|DELETE /rules/{id} POST
/api/browser-ai/rules/test-bot test an AI Guard Bot rule POST /api/browser-ai/rules/generate-regex generate
regex from a policy description GET|PUT /api/browser-ai/controls interaction controls (single row) GET
/api/browser-ai/targets + POST /targets, POST /targets/import, PUT|DELETE /targets/{id}
```

Fleet / agents / setup

```
GET /api/browser-ai/agents list agents (filters: status, source, search) POST
/api/browser-ai/agents/heartbeat agent registration + health (Guard secret) POST
/api/browser-ai/agents/bulk-delete DELETE /api/browser-ai/agents/{id} PUT
/api/browser-ai/agents/{id}/contact-email GET /api/browser-ai/agents/settings GET|PUT
/api/browser-ai/agents/uninstall-key company uninstall key GET /api/browser-ai/agents/{id}/uninstall-key
POST /api/browser-ai/agents/{id}/uninstall-key/rotate POST /api/browser-ai/agents/uninstall-verify employee
key check POST /api/browser-ai/agents/uninstall agent requests removal POST
/api/browser-ai/agents/uninstall-ack POST /api/browser-ai/agents/uninstall-status POST
/api/browser-ai/agents/{id}/remote-uninstall admin-initiated removal GET|PUT /api/browser-ai/fleet-config
company-wide Guard defaults GET /api/browser-ai/setup/info GET /api/browser-ai/setup/download.zip aliases:
-windows.zip, -mac.zip POST /api/browser-ai/setup/rebuild rebuild + republish Guard packages GET
/api/browser-ai/setup/rebuild-history GET /api/browser-ai/setup/proxy-bundle.json GET
/api/browser-ai/setup/proxy-bundle.zip hot-update bundle for installed Guards POST
/api/browser-ai/send-warning-email
```

Ingest / PAC / helpers

```
POST /api/browser-ai/intercept agent prompt/upload event POST /api/browser-ai/intercept-file agent file
event GET /api/browser-ai/attachments/{id} attachment download (auth-gated) GET /api/browser-ai/proxy.pac
PAC script GET /api/browser-ai/pac PAC script (alias used by GPO / network proxy) GET
/api/browser-ai/ollama-models models available for AI Guard Bot rules
```

9.2 Guard-fleet authentication (no user session)

isPublicBrowserAIRoute and **isGuardKeyBrowserAIRoute** (**handlers/middlewares.go**) let the Guard agent call the backend **without a dashboard session cookie**:

- **Public (secret-validated):** **intercept**, **intercept-file**, **search-logs** (POST), **proxy.pac**, **pac**, **agents/heartbeat**, **agents/uninstall-verify**, **agents/uninstall**, **agents/uninstall-ack**, **agents/uninstall-status**.
- **Guard-key:** **targets**, **rules**, **controls**, **fleet-config** (GET), **setup/download*.zip**, **setup/proxy-bundle.json**, **setup/proxy-bundle.zip**.
- The fleet secret is **UNIFAI_GUARD_SECRET**. With **UNIFAI_GUARD_REQUIRE_SECRET=1** (production default in **.env.example** and compose) agent APIs **fail closed** when the secret is missing or wrong (**browser_ai_guard_security.go**).
- **UNIFAI_PAC_ALLOW_QUERY_PROXY=0** (default) makes the PAC ignore a client-supplied **?proxy=** and use only fleet/env values - anti PAC-hijack.

9.3 Guard desktop agent (apps/browser-guard/agent)

unifai_agent.py is the entry point. Supporting modules:

Module	Responsibility
guard_bootstrap.py	single-instance lock, logging bootstrap, runtime version resolution
agent_config.py	env -> JSON config precedence (UNIFAI_BACKEND_URL , UNIFAI_PROXY_ADDR , UNIFAI_PAC_URL , UNIFAI_PAC_SYNC_SECONDS , UNIFAI_SERVER_MODE , UNIFAI_AGENT_TYPE , UNIFAI_LISTEN_HOST , UNIFAI_PAC_ADVERTISE_ADDR , UNIFAI_AGENT_ID , UNIFAI_AGENT_HOSTNAME)
agent_identity.py	persistent agent ID + host/OS metadata for the fleet inventory
agent_heartbeat.py	periodic heartbeat, fleet config pull, remote-uninstall intent, health detail
agent_health.py	port checks (port_open , free_proxy_port), health loop
agent_pac_content.py	backend reachability check, PAC fetch, local PAC write
agent_pac_server.py	local PAC/health HTTP server (PAC_HTTP_PORT , default 18195, falls back to +1/+2 if busy)
agent_pac_orchestration.py	apply PAC with cache-bust, clear/restore runtime, fail-open direct mode
agent_proxy_engine.py	runs the mitmproxy worker child process
agent_proxy_bundle.py	SHA-tracked hot-update bundle, mark-bad, relaunch, running SHA
agent_certs.py	installs the mitmproxy CA into the OS trust store, writes ca_install_status.txt
agent_browser_policy.py	browser QUIC / DoH policy adjustments
agent_autoupdate.py	compares UNIFAI_GUARD_RUNTIME_VERSION with the installer version and updates
agent_lifecycle.py	first-run prompt, uninstall flow (key-gated), user messages
agent_logging.py	file logging, get_resource_path
agent_http.py , agent_state.py , guard_platform.py	HTTP client, local state, OS paths (data_dir , log_hint_path , register_autostart)

Default paths: data %LOCALAPPDATA%\UnifAI\Guard (Windows) /

~/Library/Application Support/UnifAI/Guard (macOS); log **unifai_guard.log**;

CA status **ca_install_status.txt**.

9.4 mitmproxy addon (apps/browser-guard/proxy/browser_ai_proxy.py)

The addon is **not** a normal Python package. **unifai_proxy_parts/MANIFEST.txt** lists the parts that are loaded **in order into one shared namespace** (preserving the original single-file monolith behaviour):

```
config_caches_rules.py config, caches, target/rule fetch + matching helpers_prompts.py prompt extraction +
helper utilities uploads_detect.py upload / file detection file_policy.py per-file-type policy decisions
extract_office_backend.py Office/PDF/image text extraction + backend calls responses_inject.py response
rewriting / block injection (SSE aware) responses_addon.py response learning (e.g. filename discovery)
```

Flow: target-domain match -> extract prompt/attachment -> evaluate rules (regex locally; **ai_bot** via the

configured LLM/Ollama) -> verdict **Allowed | Redacted | Warned | Blocked | SiteBlocked | Bot Answered**

-> inject the browser-visible response (JSON error, Anthropic SSE **unifai_guard_blocked**, etc.)

-> POST the event to **/api/browser-ai/intercept**.

Editing any part requires rebuilding the Guard (**installer/build_installer.bat** on Windows, **installer/build_macos.sh** on macOS) and keeping cross-part names intact.

9.5 Guard distribution and hot update

- Build: **make build-guard-windows** (Inno Setup) / **make build-guard-mac** (+ **-pkg**) - see **apps/browser-guard/README.md**. Without **make**, run **installer\build_installer.bat**.
- Config sync: **python apps/browser-guard/scripts/sync_config_from_env.py** regenerates **config/unifai_guard_config.json** and the docs from **.env** (**SERVER_DOMAIN**, **UNIFAI_PROXY_ADDR**, **PAC_HTTP_PORT**, **UNIFAI_GUARD_SECRET**).
- Server-side artifacts live in **apps/browser-guard/release/** and are copied into the image (**/app/release**), so **Browser AI -> Setup -> Download Setup ZIP** serves them.
- **Browser AI -> Setup -> Rebuild & Publish** re-publishes the proxy/agent Python sources as a bundle (**/api/browser-ai/setup/proxy-bundle.zip**); installed agents pick it up via **agent_proxy_bundle.py**.
- Version file: **apps/browser-guard/release/VERSION.txt** -> **1.1.15**.

10. Frontend (ui/)

10.1 Build and integration

- Vite + React 19 + TanStack Router (file-based routes under **ui/app/****, generated into **ui/app/routeTree.gen.ts**).
- State: Redux Toolkit with RTK Query APIs under **ui/app/_fallbacks/enterprise/lib/store/apis/*** (**accessProfileApi**, **alertChannelsApi**, **auditLogsApi**, **businessUnitsApi**, **circuitBreakerApi**, **clusterApi**, **connectorsApi**, **loadBalancerApi**, **mcpToolGroupsApi**, **promptDeploymentsApi**, **rbacApi**, **scimApi**, **virtualKeyUsersApi**, ...).
- **npm run build:unifai** = rebrand + **vite build** + typecheck; **npm run copy-build** copies **out/** into **transports/unifai-http/ui**, which the Go binary embeds (**//go:embed**). The UI therefore ships **inside** the server binary - no separate web server is required.
- Dev server: **npm run dev**; **make dev** runs UI + API together.
- Quality gates: **npm run typecheck** (tsc --noEmit), **npm run lint** (oxlint), **npm run format** (oxfmt).

10.2 Pages

Top level: **/** (landing), **/login**, **/signup**, **/oauth/consent**, **/pprof**, **/workspace/****, plus **__root**, **__notFound**, **__error**.

Workspace pages (one **page.tsx** per route, each with a **layout.tsx**):

Section	Routes
Observability	dashboard, logs, logs/connectors, mcp-logs, observability, audit-logs, alert-channels
Browser AI	browser-ai (tabs: overview, logs, search-logs, rules, targets, agents, telemetry, setup)
Models	providers, providers/model-limits, providers/routing-rules, model-catalog, model-limits, custom-pricing, custom-pricing/overrides, complexity-router, routing-rules, routing-rules/tree, circuit-breaker
MCP Gateway	mcp-gateway, mcp-registry, mcp-registry/library, mcp-registry/oauth-callback, mcp-tool-groups, mcp-sessions (+ auth, auth-failed, auth-success), oauth-grants, mcp-auth-config, mcp-settings
Governance	governance, governance/virtual-keys, governance/users, governance/teams, governance/business-units, governance/customers, governance/rbac, governance/access-profiles, rbac, virtual-keys, scim, scim/oauth-discover-callback
Guardrails	guardrails, guardrails/configuration, guardrails/providers
Platform	plugins, cluster, adaptive-routing (+ settings), prompt-repo (+ prompts), skills-repo, docs, config (+ api-keys, caching, client-settings, compatibility, feature-flags, logging, mcp-gateway, observability, performance-tuning, pricing-config, proxy, security)

10.3 Sidebar and section gating

ui/components/sidebar.tsx builds the menu from **ui/lib/constants/workspaceSections.ts**:

```
observability | browser-ai | models | mcp-gateway | plugins | governance | guardrails | cluster-config |
adaptive-routing | prompt-repository | skills-repository | settings
```

Each section has a **defaultPath** and optional child **items**. Visibility is decided by:

1. RBAC hooks - **useRbac(RbacResource.X, RbacOperation.View)**.
2. Role checks for admin-only pages (Users requires **admin** or **sub_admin**).
3. Section grants for section-scoped sessions (**scopedSidebarSections** from **authStatus.allowed_sections**), with automatic redirect to **getDefaultPathForSections(...)**

when the user lands on a route they are not granted.

Browser AI is reachable with **Logs** view RBAC or the **admin/sub_admin** role

(**hasBrowserAiAccess**), matching the backend mapping of **/api/browser-ai/*** -> **Logs**.

10.4 Browser AI UI implementation files

ui/app/workspace/browser-ai/: **page.tsx** (tabs, tables, dialogs), **browserAiTypes.ts**, **browserAiConstants.ts** (**GUARD_BOT_OLLAMA_PROVIDER** = "ollama", **GUARD_BOT_OLLAMA_MODEL** = "llama3.2", reference image max 512 KB), **browserAiFormat.ts**, **browserAiLogHelpers.ts**, **browserAiGuardHelpers.ts**, **guardBotModelPickers.tsx**, **guardRuleAIEvaluatorFields.tsx**, **guardRuleImportDialog.tsx**, **targetImportDialog.tsx**, **regexLiveTestPanel.tsx**, **logPromptPreviewCell.tsx**, **logTimestampCell.tsx**, **logBadges.tsx**, **attachmentPreview.ts**, **relatedHosts.ts**, plus import templates under **examples/**.

Types as implemented: **GuardRuleAction** = "BLOCK" | "REDACT" | "WARN",

GuardRuleSeverity = "CRITICAL" | "HIGH" | "MEDIUM", GuardRuleEvalMode = "ai" | "regex".

11. Security model

Area	Implementation
Dashboard sessions	Session rows stored in DB; token cookie with HttpOnly + SameSite=Lax (+ Secure only for real TLS or a trusted proxy sending X-Forwarded-Proto: https); default 24 h TTL
WebSocket auth	Cookie-only (/api/session/ws-ticket issues short-lived tickets); legacy ?token= query auth removed (token-leak risk)
Inference auth	Virtual Key or provider key via Authorization: Bearer / x-api-key ; enforce_auth_on_inference flag
Brute force	Per-IP: 30 failed logins / 15 min window; per-username lockout rows in the DB; remaining minutes returned to the client
Password reset	PASSWORD_RESET_SECRET (min 32 chars) HMAC-signed token, 10 min expiry; 5 requests/hour/target + 10/hour/IP; OTP max 5 attempts; 2 min cooldown
Registration	Self-signup with e-mailed code (email_unverified) -> admin approval (pending -> approved) or rejected ; SCIM can set disabled
Authorization	Two RBAC layers (resource×operation, section grants) + audit middleware recording mutating /api/* actions
Secrets at rest	framework/encrypt encryption for DB-stored secrets when UNIFAI_ENCRYPTION_KEY is set (provider keys, alert channels, uninstall keys)
Guard fleet	Shared UNIFAI_GUARD_SECRET , fail-closed with UNIFAI_GUARD_REQUIRE_SECRET=1 ; PAC hijack protection via UNIFAI_PAC_ALLOW_QUERY_PROXY=0
Guard uninstall	Company and per-agent uninstall keys, stored hashed + encrypted, rotatable, with an audit trail and remote-uninstall flow
Transport	TLS terminated at the edge/reverse proxy; TRUST_PROXY_HEADERS=1 honours X-Forwarded-Proto/X-Forwarded-For only from trusted peers (TRUSTED_PROXIES required for public edges)
CORS	client.allowed_origins (from SERVER_DOMAIN); optional ALLOW_LOCALHOST_CORS=1 for a local UI against a remote API
Headers	SecurityHeadersMiddleware applied to every response
Open bootstrap	When auth is disabled, unauthenticated admin is allowed only from loopback or with ALLOW_OPEN_AUTH=1 (local bootstrap only)
Inference hardening	Provider headers normalised (x-api-key removed), password-as-Bearer removed, error sanitisation (errorsanitizer.go)
Destructive ops	Admin-only guards (adminOnlyLogs , requireGuardAdmin); attachment access privileged

12. Build, test and developer workflow

12.1 Gateway

```
make setup-workspace # go work use ./core ./framework ./plugins/... ./transports make dev # UI (vite) + API
with hot reload (air) make build # build UI then the Go binary make run # build + run make fmt ; make lint #
gofmt / linter make test # unifai-http transport tests (gotestsum) make test-core # core provider tests
(PROVIDER=openai TESTCASE=... PATTERN=...) make test-framework # framework tests make test-plugins # plugin tests
make test-http-transport # HTTP transport tests make test-governance # governance tests make test-mcp # MCP
tests (TYPE=... TESTCASE=...) make test-semantic-cache # semantic cache e2e make test-all # all of the above make
run-e2e ; make run-e2e-ui ; make run-e2e-headed ; make run-e2e-api make test-integrations-py ; make
test-integrations-ts make run-provider-harness-test ; make run-cli-harness-test
```

Reports: **make generate-html-reports**; cleanup with **make clean-test-reports**.

12.2 UI

```
cd ui npm ci npm run dev # Vite dev server npm run typecheck # tsc --noEmit npm run lint # oxlint npm run
build # vite build + typecheck + copy into transports/unifai-http/ui
```

12.3 Guard

```
make sync-guard-config # regenerate Guard config/docs from .env make build-guard-windows # Windows: Setup.exe
+ Windows ZIP (Inno Setup 6 required) make build-guard-mac # macOS: .app + macOS ZIP (must run on a Mac) make
build-guard-mac-pkg # macOS: .pkg from the existing .app make help-guard # list Guard targets
```

Guard unit tests live in **apps/browser-guard/agent/tests/** and **apps/browser-guard/proxy/tests/**

(PAC targets, proxy bundle, file-type extraction, latency, rule actions, target predict, upload names).

12.4 Docker / Helm

```
make docker-image # LOCAL=1 uses deploy/docker/Dockerfile.local (go workspace based) make docker-run # run
the container (CONFIG=... optional) make helm-index # repackage chart + regenerate deploy/helm/index.yaml
```

12.5 Conventions

- Go modules resolve through **go.work**; add new modules with **go work use**.
- Handlers keep the **RegisterRoutes(router, middlewares...)** pattern and are grouped by feature.
- UI files use camelCase names, Radix primitives from **components/ui**, and RTK Query for API calls.
- Guard proxy parts must stay in **MANIFEST.txt** order.

13. Extension points

Goal	Where to work
Add an LLM provider	new package under core/providers/<name> implementing the provider contract, then register it in the provider registry
Add a pass-through API surface	new file in transports/unifai-http/integrations/ + registration in integrations/router.go
Add a plugin	new module under plugins/<name> exposing PluginName ; add a case in loadBuiltinPlugin (server/plugins.go) and add the module to go.work
Add a dashboard API	new handlers/<feature>.go with RegisterRoutes , then call it from server.RegisterAPIRoutes
Gate a new route with RBAC	extend PathRequirementFor (framework/rbac/enforce.go) and, for section-scoped sessions, SectionRequirementFor (framework/rbac/sections.go)

Goal	Where to work
Add an RBAC resource	add it to RBACResourceNames (framework/configstore/workspace.go); permissions are generated from the cartesian catalog
Add a UI page	create ui/app/workspace/<route>/page.tsx + layout.tsx , add the section item in ui/lib/constants/workspaceSections.ts , and the nav entry in ui/components/sidebar.tsx
Add a Guard detection	edit the relevant part in apps/browser-guard/proxy/unifai_proxy_parts/ (keep MANIFEST order) and rebuild the Guard
Add a Guard rule type	BrowserGuardRule (framework/logstore/browser_ai.go) + the UI rule editor + the proxy evaluator
Change gateway guardrail behaviour	plugins/guardrails + /api/guardrails/*

14. Glossary

Term	Meaning in this codebase
Virtual Key (VK)	A gateway-issued credential that maps to budgets/limits/teams/customers and is used by inference clients
ConfigStore	Config/workspace database (users, roles, keys, providers, settings)
LogsStore	Log database (LLM logs, MCP logs, Browser AI logs, aggregates)
Section grant	Sidebar-section permission (governance/users) used for section-scoped sessions
DAC	Data access scope on a role: all-data or own-data
Target website	A monitored AI domain (with platform name, host role and monitored/blocked flags)
Guard rule	A detection rule: regex (pattern) or ai_bot (LLM-judged, optional reference image)
Guard agent	A Guard installation reporting to the fleet (endpoint or network)
Fleet config	Company-wide Guard defaults that agents pull on heartbeat
PAC	Proxy auto-config script served by the backend, pointing browsers at the Guard proxy
Proxy bundle	Server-published hot-update bundle of the Guard proxy/agent Python code
Uninstall key	Company/per-agent secret required to turn the Guard off
Intercept	The Guard -> backend event that records a prompt, upload or verdict

UnifAI - Server & Deployment Documentation

Audience: DevOps, IT administrators, SRE

Scope: requirements, environment, deployment (Docker compose / Helm), database, TLS, operations, upgrades, troubleshooting, hardening

Baseline: repo commit **16d4c3e**

1. What you are deploying

A single container/binary serves everything:

Served by the same process	Path/port
Admin + user dashboard (embedded React SPA)	/, /workspace/**
Management API	/api/**
AI inference API (OpenAI-compatible)	/v1/**
Provider pass-through APIs	/openai/**, /anthropic/**, /genai/**, /bedrock/**, ...
Browser AI guard control plane + agent ingestion	/api/browser-ai/**
WebSocket / realtime	/ws, /v1/realtime
Health	/health

Optional second container (compose profile **network-proxy**) runs the mitmproxy-based corporate proxy for office/GPO deployments.

2. Requirements

Component	Minimum	Recommended (production)
Container host	Linux with Docker 24+ / compose v2	Ubuntu 22.04+ / RHEL 9
CPU	2 vCPU	4-8 vCPU
RAM	2 GB	8-16 GB
Disk	10 GB	50 GB+ (logs + attachments + Guard packages)
Database	Postgres 13+	Postgres 15+ with TLS and automated backups
Outbound network	HTTPS to AI provider APIs	plus a corporate proxy if required
Ollama (only for AI Guard Bot rules)	reachable HTTP endpoint	dedicated CPU/GPU host
Reverse proxy	nginx / Caddy / HAProxy / Cloudflare	TLS 1.2+, HSTS
Build toolchain (only to build images)	Go 1.26+, Node 25+, Docker	-

Ports used by the stack (from **.env.example**):

Variable	Default	Purpose
APP_PORT	6000 (binary default 8001)	UnifAI HTTP / UI / API
PROXY_PORT	18182	Docker network proxy (profile network-proxy)
UNIFAI_PROXY_ADDR	127.0.0.1:18103	Laptop Guard local MITM bind
PAC_HTTP_PORT	18195	Laptop Guard local PAC/health server
MITM_WEB_PORT	18183	Dev-only mitmweb UI
UI_PORT	3100	Vite dev server (development only)

3. Environment variables

Copy **.env.example** -> **.env** and fill in real values. **.env** is git-ignored and should be the **only** place live secrets exist.

3.1 Server / network

Variable	Required	Notes
SERVER_DOMAIN	yes	Public base URL, e.g. https://unifai.example.com . Drives CORS origins, MCP external URL, Guard config and PAC URLs. Compose fails fast if unset
APP_PORT	yes	Listening port
APP_HOST	no	Bind address (compose uses 0.0.0.0)
CONTAINER_NAME	yes	Container/hostname used for inter-container calls (raksha_tech)
PROXY_PORT	no	Docker network proxy port
DOCKER_NETWORK	no	Bridge network name (default unifai-network)
OLLAMA_DOCKER_NETWORK	no	External network containing the Ollama container (default 1panel-network)

3.2 Database

Variable	Required	Notes
DB_TYPE	yes	postgres (prod default), mysql or sqlite
DB_HOST , DB_PORT , DB_NAME , DB_USER , DB_PASSWORD	yes	Compose fails fast if unset
DB_SSL_MODE	no	disable locally; use require/verify-full in production

3.3 Auth, admin and crypto

Variable	Required	Notes
ADMIN_EMAIL , ADMIN_PASSWORD	yes	Bootstrap admin (change immediately after first login)
UNIFAI_ENCRYPTION_KEY	production	Long random secret; encrypts secrets stored in the DB
UNIFAI_GUARD_SECRET	production	Fleet secret shared by Guard agents (openssl rand -base64 32)

Variable	Required	Notes
UNIFAI_GUARD_REQUIRE_SECRET	production (1)	Fail-closed when the Guard secret is missing/wrong
UNIFAI_PAC_ALLOW_QUERY_PROXY	no (0)	0 = PAC ignores a client ?proxy= (anti hijack)
PASSWORD_RESET_SECRET	yes	≥ 32 chars (openssl rand -base64 48); without it password reset returns HTTP 500
TRUST_PROXY_HEADERS	no (1)	Honour X-Forwarded-* only from trusted peers
TRUSTED_PROXIES	public edge	Comma-separated IPs/CIDRs of the edge proxy
CLUSTER_REPLICATE_SECRET	multi-node	Required by POST /internal/cluster/kv
ALLOW_LOCALHOST_CORS	no	1 allows credentialed CORS from localhost
ALLOW_OPEN_AUTH	no	1 allows unauthenticated admin while auth is disabled (local bootstrap only)

3.4 Logging, data and branding

Variable	Notes
LOG_LEVEL	debug / info / warn / error (default info)
LOG_STYLE	json (default) or pretty
APP_DIR	Data directory holding config.json , logs/ , pdf/ , attachments/ (default /app/data)
UNIFAI_COMPANY_NAME , UNIFAI_COMPANY_LOGO	Dashboard branding
OLLAMA_URL / BROWSER_AI_OLLAMA_URL	Ollama endpoint used by AI Guard Bot rules
UNIFAI_BACKEND_URL	Backend URL for the UI dev server and Guard

3.5 Guard agent (client side)

Variable	Notes
UNIFAI_BACKEND_URL / SERVER_DOMAIN	Backend the agent reports to
UNIFAI_PROXY_ADDR	Local MITM bind (127.0.0.1:18103)
UNIFAI_PAC_URL	PAC endpoint (defaults to <backend>/api/browser-ai/pac)
UNIFAI_PAC_SYNC_SECONDS	PAC/rule refresh interval (default 3, clamped 2-600)
UNIFAI_SERVER_MODE	1 for network/server proxy mode
UNIFAI_AGENT_TYPE	endpoint or network
UNIFAI_LISTEN_HOST	127.0.0.1 (endpoint) or 0.0.0.0 (network)
UNIFAI_PAC_ADVERTISE_ADDR	Address advertised in PAC for network mode
UNIFAI_AGENT_ID , UNIFAI_AGENT_HOSTNAME	Fleet identity
UNIFAI_NETWORK_AGENT_REGISTER	1 lets a network proxy register itself in the fleet
UNIFAI_FAIL_OPEN	1 = allow traffic if the backend is unreachable (use with care)
UNIFAI_EVAL_TIMEOUT	AI Guard Bot evaluation timeout in seconds

After changing `.env`, regenerate the Guard config:

```
python apps/browser-guard/scripts/sync_config_from_env.py
```

4. config.json reference

The runtime config is `APP_DIR/config.json` (`/app/data/config.json` inside the container). On first start the entrypoint copies `/app/configs/config.json` there. It supports `env.NAME` indirection so secrets are not stored in the file.

Structure observed in `configs/config.json`:

Key	Purpose
config_store	Config DB: <code>enabled</code> , <code>type</code> (<code>postgres/mysql/sqlite</code>), <code>config</code> . <code>{host,port,db_name,user,password,ssl_mode}</code> (all via <code>env.DB_*</code>)
logs_store	Log DB - same shape as <code>config_store</code>
client	<code>drop_excess_requests</code> , <code>initial_pool_size</code> (5000), <code>enable_logging</code> , <code>disable_content_logging</code> , <code>log_retention_days</code> (365), <code>enforce_auth_on_inference</code> , <code>allowed_origins</code> (<code>env.SERVER_DOMAIN</code>), <code>max_request_body_size_mb</code> (100), <code>mcp_external_client_url</code> , <code>compat.should_convert_params</code> , <code>disable_db_pings_in_health</code>
governance	<code>auth_config.is_enabled</code> , <code>auth_config.admin_username</code> (<code>env.ADMIN_EMAIL</code>), <code>admin_password</code> (<code>env.ADMIN_PASSWORD</code>)
framework	<code>pricing</code> , plus framework-level feature configuration
plugins	Per-plugin config and enable/disable + ordering
providers	Provider definitions, keys and network config
mcp, vector_store, oauth2, etc.	Optional subsystems

Also under `configs/`: `mcp-library.json` (MCP catalog seed), `model-parameters.json`, `pricing.json`.

Config is loaded into memory at boot; changes made through the UI are persisted to the DB and to

`config.json`. Do **not** bind-mount `config.json` as a single file - mount the whole `./data`

directory, otherwise atomic renames fail ("device or resource busy"). The entrypoint explicitly

errors out if `config.json` is not writable.

5. Docker compose deployment (recommended path)

5.1 Prepare

```
git clone <repo> && cd unifai_project cp .env.example .env # edit .env: SERVER_DOMAIN, DB_*, ADMIN_*,
UNIFAI_ENCRYPTION_KEY, # UNIFAI_GUARD_SECRET, PASSWORD_RESET_SECRET
```

5.2 Build and start

```
docker compose up -d --build # builds the UI + Go binary from source docker compose logs -f raksha_tech #
watch startup (plugin status table, listener line)
```

The build is multi-stage (**deploy/docker/Dockerfile.local**):

node:25-alpine builds the React UI -> **golang:1.26.4-alpine** builds the binary with a Go workspace

-> **alpine:3.23** runtime with **musl libgcc ca-certificates zlib git**, non-root **appuser**, a healthcheck on **/health**, and the Guard release artifacts copied to **/app/release**.

5.3 Volumes and mounts (from **docker-compose.yml**)

Mount	Purpose
./data:/app/data	Runtime state: config.json , logs, attachments, PDFs
./configs:/app/configs	Default configs
./apps/browser-guard/release:/app/release	Guard install packages served by Setup downloads
./apps/browser-guard/proxy:/app/guard-proxy:ro	Guard proxy source for Rebuild & Publish
./apps/browser-guard/agent:/app/guard-agent:ro	Guard agent source for Rebuild & Publish

extra_hosts: host.docker.internal:host-gateway lets the container reach host services (e.g. Ollama);

DNS is pinned to **8.8.8.8 / 1.1.1.1**; the container joins both the default bridge network and the external Ollama network (**1panel-network** by default - create it once if it does not exist).

5.4 Optional network proxy (office / GPO)

```
docker compose --profile network-proxy up -d raksha_browser_ai_proxy
```

This runs **mitmproxy/mitmproxy:latest** with **mitmdump -s /app/proxy/browser_ai_proxy.py -p PROXY_PORT**

and environment **UNIFAI_BACKEND_URL=http://raksha_tech:APP_PORT**, **UNIFAI_SERVER_MODE=1**, **UNIFAI_AGENT_TYPE=network**, **UNIFAI_FAIL_OPEN=0**, **UNIFAI_GUARD_SECRET=...**

Point office PAC/GPO to:

```
https://<SERVER_DOMAIN>/api/browser-ai/pac?proxy=<proxy-host>:18182
```

and distribute the mitmproxy CA to client machines for HTTPS inspection.

Never MITM the same browser session with both the laptop Guard and the corporate proxy.

5.5 Verify

```
curl -fsS https://<SERVER_DOMAIN>/health curl -fsS https://<SERVER_DOMAIN>/api/browser-ai/proxy.pac curl -fsS https://<SERVER_DOMAIN>/api/browser-ai/setup/info
```

Expect JSON (or a PAC script), never the HTML login page - HTML means the backend build does not include the Browser AI routes.

6. Database setup and migrations

6.1 Postgres (production)

```
CREATE DATABASE unifai_new; CREATE USER agent_unify WITH ENCRYPTED PASSWORD '<strong-password>'; GRANT ALL PRIVILEGES ON DATABASE unifai_new TO agent_unify; \c unifai_new GRANT ALL ON SCHEMA public TO agent_unify;
```

Then set **DB_TYPE=postgres**, **DB_HOST**, **DB_PORT=5432**, **DB_NAME**, **DB_USER**, **DB_PASSWORD**, **DB_SSL_MODE=require**.

6.2 Automatic schema management

- On boot the server connects via GORM, creates/updates tables and seeds defaults: RBAC catalog, three system roles, the single-row Browser AI control settings, and any missing config rows.
- framework/migrator** (**addcolumn.go**, **dropcolumn.go**) and **configstore/migrations.go** / **logstore/migrations.go** handle additive schema changes. Migrations are safe to run repeatedly.
- framework/configstore/tables/migrations** run for both the config DB and the log DB.

6.3 Tables you will care about in operations

Area	Tables
Identity	governance_users , sessions , rbac_roles , access_profiles , audit_logs
Governance	virtual keys + their link tables, teams, customers, business units, budgets, rate limits
Provider config	provider rows + keys, models, model configs, pricing, pricing overrides
Logs	LLM log tables, mcp_logs , dashboard materialized views
Browser AI	browser_ai_logs , browser_ai_search_logs , browser_guard_rules , browser_ai_target_websites , browser_ai_control_settings , browser_ai_agents , browser_ai_agent_settings , browser_guard_fleet_config , browser_guard_rebuild_logs , browser_ai_warning_emails
Ops	browser_guard_rebuild_logs , distributed-lock rows, feature flags, plugins

6.4 Retention

client.log_retention_days (default 365) drives the log-retention cleaner started during bootstrap;

0 disables the cleaner. Async inference jobs are cleaned by the async-job cleaner.

Browser AI attachments are disk files under **APP_DIR/attachments** with DB expiry metadata - the retention job and manual deletes clean them.

7. Reverse proxy and TLS

Terminate TLS at the edge; the container serves plain HTTP internally.

nginx example

```
server { listen 443 ssl http2; server_name unifai.example.com; ssl_certificate /etc/ssl/fullchain.pem;
ssl_certificate_key /etc/ssl/privkey.pem; client_max_body_size 100m; # match client.max_request_body_size_mb
location / { proxy_pass http://127.0.0.1:6000; proxy_http_version 1.1; proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme; # WebSocket / SSE proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade"; proxy_read_timeout 3600s; proxy_buffering off; # required for
streaming responses } }
```

Important:

- Set **TRUST_PROXY_HEADERS=1** and **TRUSTED_PROXIES=<edge IP/CIDR>** so **X-Forwarded-*** is honoured (this controls the cookie **Secure** flag and client IP for rate limits/logs).
- Disable proxy buffering for **/v1/*** so SSE streaming is not buffered.
- Cloudflare or any CDN counts as a proxy - list its egress ranges in **TRUSTED_PROXIES**.

8. Kubernetes / Helm

Chart: **deploy/helm/unifai** (**Chart.yaml**, **values.yaml**, **values.schema.json**, templates for deployment, service, ingress, HPA, secrets, configmap, service account/RBAC, and optional backing services: Postgres, Redis, Qdrant, Weaviate).

```
cd deploy/helm ./scripts/generate-values.sh # write a values file for your environment ./scripts/validate.sh
# schema/sanity validation ./scripts/install.sh # helm install # or manually: helm install unifai ./unifai -f
my-values.yaml
```

Values examples provided: **sqlite-only**, **sqlite-redis**, **sqlite-qdrant**, **sqlite-weaviate**, **postgres-only**, **postgres-redis**, **postgres-qdrant**, **postgres-weaviate**, **mixed-backend**, **external-postgres**, **production-ha**, **providers-and-virtual-keys**, **secrets-from-k8s**.

For production HA:

- Use **production-ha.yaml** as a starting point (replicas + HPA + external Postgres).
- Set **CLUSTER_REPLICATE_SECRET** so nodes can replicate config through **POST /internal/cluster/kv**.
- Prefer a managed/external Postgres with TLS and backups over the in-chart Postgres.
- Set an Ingress with TLS and **proxy-body-size** ≥ 100 MB; keep SSE working (disable buffering).

Repackage/regenerate the chart index after value/template changes: **make helm-index**.

9. Operations

9.1 Start, stop, restart

```
docker compose up -d --build # build + start
docker compose restart raksha_tech # restart app only
docker compose stop # stop all
docker compose down # remove containers (data stays in ./data and the DB)
docker compose ps # status
```

Graceful shutdown: the binary handles SIGINT/SIGTERM, closes realtime sessions, shuts down the core client and storage engines with a 30 s timeout, and also drains the pprof server (90 s).

9.2 Logs

- Container logs: **docker compose logs -f --tail=200 raksha_tech** (JSON by default; **LOG_STYLE=pretty** for humans, **LOG_LEVEL=debug** when diagnosing).
- Startup line to look for: **successfully started unifai, serving UI on http://0.0.0.0:<port>**, plus the plugin status table (**plugin status: <name> - <status>**).
- App file logs under **APP_DIR/logs/**.
- Request/business logs are in the dashboard (LLM Logs, MCP Logs, Browser AI -> Prompt Logs) and in the log database - not only in stdout.

9.3 Health and monitoring

- Liveness/readiness: **GET /health** -> 200 **{"status":"ok", ...}**; 503 lists the failing component.

The image healthcheck already calls this every 30 s.

- Watch: CPU/RAM of the container, DB connections, log-table growth, **browser_ai_logs** growth, Guard agent heartbeats (Browser AI -> Guard Agents -> last seen), rebuild history.
- Enable OpenTelemetry (**otel** plugin) and connectors (Datadog/New Relic/BigQuery/Kafka/PubSub) for external observability. Alert channels deliver webhook notifications.
- Dev/diagnostic: pprof endpoints exist only in dev builds; **/pprof** page in the UI.

9.4 Backups and restore

Data	How to back up
Postgres (ConfigStore + LogsStore)	pg_dump / managed snapshots - this is the critical asset
./data	archive the directory (config.json , attachments, PDFs, logs)
./configs	keep under version control
Guard packages	apps/browser-guard/release/ (rebuildable, but keep for consistency)
.env	store in a secret manager; never in git

Restore order: database -> **./data** -> recreate the container with the same **.env**.

Recommended: nightly **pg_dump** with 30-day retention, weekly full **./data** archive, and an off-site copy of **.env/secrets**.

9.5 Upgrade procedure

1. Back up the DB and **./data**.
2. **git pull** (and **make sync-guard-config** if Guard settings changed).
3. Rebuild and restart: **docker compose up -d --build**.
4. Watch startup logs; schema migrations and RBAC seeding run automatically.
5. Verify **GET /health**, log in, open Browser AI (targets, rules, agents), and send a test prompt through a provider.
6. Rollback: check out the previous commit, rebuild, restore the DB dump if a schema change is incompatible.

Perf-critical rebuild: the Docker build compiles the UI (Node) then the Go binary - expect several minutes on first build. Docker layer cache makes subsequent builds much faster.

9.6 Scaling and HA

- The app is stateless apart from the DB and **APP_DATA** - run multiple replicas behind a load balancer.
- Multi-node config replication needs **CLUSTER_REPLICATE_SECRET**; see Workspace -> Cluster.
- Postgres is the shared bottleneck: use a managed instance, connection pooling, and monitor slow queries.
- Use the adaptive/load-balancer routing (**/api/load-balancer**) and circuit breaker policies to spread traffic across providers/keys and isolate failing ones.
- Guard agents heartbeat to any replica (the fleet is stored in the DB).

9.7 Guard fleet operations (server side)

- Setup -> Download Setup ZIP (**/api/browser-ai/setup/download[-windows|-mac].zip**) serves the packages in **apps/browser-guard/release/**.
- Setup -> Rebuild & Publish (**POST /api/browser-ai/setup/rebuild**) republishes Guard Python code as a bundle; history is in **/api/browser-ai/setup/rebuild-history** and **browser_guard_rebuild_logs**.
- Uninstall key: **GET|PUT /api/browser-ai/agents/uninstall-key** (company) or per-agent rotate.
- Remote uninstall: **POST /api/browser-ai/agents/{id}/remote-uninstall**.
- Fleet config: **GET|PUT /api/browser-ai/fleet-config** - agents pull it on heartbeat.
- When you rotate **UNIFAI_GUARD_SECRET**, re-sync Guard config and re-publish packages; existing agents keep working only if their config is updated.

10. Troubleshooting

Symptom	Likely cause	Fix
Compose refuses to start: set SERVER_DOMAIN / DB_HOST / ADMIN_EMAIL ... in .env	Required variable missing	Fill the variable in .env - compose uses fail-fast substitution
Error: /app/data/config.json is not writable	config.json mounted as a single file	Mount only ./data:/app/data

Symptom	Likely cause	Fix
<code>/api/browser-ai/*</code> returns the login page HTML	Image lacks Browser AI routes, or an edge rule intercepts the path	Deploy the current backend; exclude <code>/api/browser-ai/*</code> from SSO redirects and SPA fallback
<code>/health</code> returns 503 config store not available	Wrong DB credentials/host, DB down, TLS mismatch	Check <code>DB_*</code> , <code>DB_SSL_MODE</code> , and DB reachability from the container
Agents never appear under Guard Agents	Wrong backend URL in Guard config, missing/mismatched <code>UNIFAI_GUARD_SECRET</code> , egress blocked	Re-run <code>sync_config_from_env.py</code> , verify <code>/api/browser-ai/agents/heartbeat</code> returns JSON, check firewall/proxy
Browsers show certificate warnings on AI sites	MITM CA not trusted on the endpoint	Check <code>ca_install_status.txt</code> ; re-trust the CA; verify GPO/MDM distribution
Prompts are not appearing in Prompt Logs	Target domain not monitored, PAC not applied, agent not running	Add/verify the Target Website, confirm PAC is applied, check <code>unifai_guard.log</code>
Rules do not take effect immediately	Rule cache (~2 s) + PAC sync interval (default 3 s)	Wait a few seconds or lower <code>UNIFAI_PAC_SYNC_SECONDS</code> (min 2)
Password reset returns 500	<code>PASSWORD_RESET_SECRET</code> missing or < 32 chars	Set a ≥ 32-char secret and restart
Login blocked: "Too many attempts..."	Per-IP (30/15 min) or per-username lockout	Wait for the window or clear the lockout rows for that key
Streaming responses arrive all at once	Reverse-proxy buffering	<code>proxy_buffering off</code> (nginx) / disable buffering
Cookie not marked Secure behind HTTPS	<code>TRUST_PROXY_HEADERS</code> off, or edge not listed	Set both so <code>X-Forwarded-Proto: https</code> is trusted
AI Guard Bot rules never evaluate	Ollama/LLM unreachable from backend or agent	Verify <code>OLLAMA_URL</code> / <code>BROWSER_AI_OLLAMA_URL</code> and <code>/api/browser-ai/ollama-models</code>
Duplicate blocks / duplicate log rows	Laptop Guard and corporate proxy both MITM the session	Use one path per network location
Container OOM or slow	Log tables + attachments grew, or heavy model-catalog fetch	Check retention and disk, raise memory, add DB pooling
MCP clients fail then recover	Expected: the health monitor reconnects	Confirm the MCP server is reachable; check MCP client status in the UI

11. Security hardening checklist

- [] Long random `UNIFAI_ENCRYPTION_KEY`, `UNIFAI_GUARD_SECRET`, `PASSWORD_RESET_SECRET`, `CLUSTER_REPLICATE_SECRET` - generated with `openssl rand -base64 32|48`, stored in a secret manager.
- [] `ADMIN_PASSWORD` changed after first login; bootstrap admin replaced by a named admin account.
- [] `UNIFAI_GUARD_REQUIRE_SECRET=1` and `UNIFAI_PAC_ALLOW_QUERY_PROXY=0`.
- [] `DB_SSL_MODE=require/verify-full`; database not publicly exposed; least-privilege DB user.
- [] TLS at the edge, HSTS, HTTP->HTTPS redirect; `TRUST_PROXY_HEADERS=1` with an explicit `TRUSTED_PROXIES` list.
- [] `client.allowed_origins` limited to your domain (no wildcard); `ALLOW_LOCALHOST_CORS` unset.
- [] `enforce_auth_on_inference` enabled; provider keys stored through encrypted governance config, never in source or compose.
- [] Least-privilege RBAC: prefer `sub_admin`/custom roles over `admin`; review

Governance -> Roles & Permissions and Access Profiles periodically.

- [] Registration approval enabled for sign-ups.
- [] SMTP configured so password reset and warning e-mails work (**/api/sntp-config**).
- [] Audit Logs reviewed regularly; alert channels configured for critical events.
- [] Backups automated and **restore-tested**; **.env** and DB dumps encrypted at rest.
- [] Guard uninstall key set, rotated periodically, shared only with IT.
- [] Guard CA distribution via GPO/MDM; macOS MDM profile

apps/browser-guard/installer/mdm/disable_private_relay.mobileconfig applied.

- [] Guard agent heartbeats monitored; investigate agents missing for more than 7 days.
- [] Rotate **UNIFAI_GUARD_SECRET** and uninstall keys on staff/IT changes.
- [] Keep the container image, Postgres and the host patched; review **Dockerfile** base-image digests.

12. Quick reference

Make targets (build/deploy)

```
make build build UI + Go binary make docker-image build image (LOCAL=1 → Dockerfile.local) make docker-run
run the container make helm-index repackage the Helm chart + regenerate index.yaml make sync-guard-config
regenerate Guard config/docs from .env make build-guard-windows Windows Guard installer + ZIP make
build-guard-mac macOS Guard app + ZIP (on a Mac) make build-guard-mac-pkg macOS .pkg (on a Mac) make docs
bundle OpenAPI + Mintlify docs dev server
```

Key files

File	Why it matters
.env / .env.example	All configuration and secrets
docker-compose.yml	Container, volumes, networks, network-proxy profile
deploy/docker/Dockerfile.local	Production image build
deploy/docker/docker-entrypoint.sh	Volume permissions, config bootstrap, argument handling
configs/config.json	Runtime config template
apps/browser-guard/config/unifai_guard_config.json	Generated Guard agent config
apps/browser-guard/release/	Guard installers served to employees
apps/browser-guard/release/VERSION.txt	Guard build version

Verification endpoints

```
GET /health GET /api/version GET /api/browser-ai/proxy.pac GET /api/browser-ai/pac?proxy=<host:port> GET
/api/browser-ai/targets GET /api/browser-ai/rules GET /api/browser-ai/fleet-config GET
/api/browser-ai/setup/info GET /api/browser-ai/agents
```

UnifAI - Administrator Guide

Audience: platform admins, IT security, compliance owners

Baseline: repo commit **16d4c3e**

1. Roles you will work with

Role	Data scope	Typical use
admin	all-data	Full control: providers and master API keys, system settings, users, RBAC, everything
sub_admin	all-data	Workspace manager: virtual keys, budgets, prompts, logs, Browser AI , MCP tool groups - but not master provider keys or system settings
user	own-data	Read/View only, on Dashboard , Logs , Inference , PromptRepository , Observability , MCPGateway . The UI label says "Prompt Repository only", but the backend also permits read-only log/observability/MCP views - narrow this with Allowed Sections (allowed_sections) so the user's sidebar matches your intent
<i>custom role</i>	configurable	Any subset of the 33 resources × 6 operations, created by an admin

Two authorisation layers apply to every API call:

1. **Resource × Operation** (e.g. **Logs: Update**, **VirtualKeys: Create**).
2. **Sidebar section grants** (**allowed_sections** on the user) for section-scoped sessions.

Important: all **/api/browser-ai/*** routes are gated by the **Logs** resource. So to let someone manage Browser AI, give them **Logs** view/update (or the **browser-ai** section grant) - **sub_admin** already has it.

2. First-run setup checklist

1. **Sign in** with the bootstrap admin from **.env** (**ADMIN_EMAIL** / **ADMIN_PASSWORD**).
2. **Change the admin password** immediately, and create a named admin account for daily use (Governance -> Users), then keep the bootstrap account for emergencies only.
3. **Configure SMTP** (**/api/smtp-config** -> Settings) so password reset, registration codes and Guard warning e-mails work.
4. **Verify secrets** are long and random: **UNIFAI_ENCRYPTION_KEY**, **UNIFAI_GUARD_SECRET**, **PASSWORD_RESET_SECRET**, **CLUSTER_REPLICATE_SECRET**.
5. **Set branding** (**UNIFAI_COMPANY_NAME**, **UNIFAI_COMPANY_LOGO**).
6. **Add providers and keys** (Models -> Model Providers) and confirm the model catalog populated.
7. **Create virtual keys** for teams/applications with budgets and rate limits.
8. **Set up Browser AI:** import Target Websites, review/import Guard Rules, set Controls and the company **uninstall key**, then distribute the Guard package from Setup.
9. **Configure alert channels** (webhooks) for the events you care about.

10. **Confirm Audit Logs** are recording your mutating actions (they are automatic).
11. **Schedule backups** (see SERVER.md §9.4) and test a restore.
12. **Verify health** - **GET /health** returns 200 and Guard agents appear under Browser AI -> Guard Agents.

3. User and access administration

3.1 Approving and managing users

Governance -> **Users** (API: **/api/session/users***):

Task	How
Approve a pending sign-up	Users -> pending user -> Approve (POST /api/session/users/{id}/approve)
Reject a request	Reject (POST /api/session/users/{id}/reject)
Create a user directly	Create user (POST /api/session/users)
Change role	Set role to user / sub_admin / admin , or assign a custom RBAC role (PUT /api/users/{id}/role)
Edit details	PUT /api/session/users/{id} - e-mail, budget, rate limit, allowed prompt repos, allowed sections
Disable / remove	DELETE /api/session/users/{id} (or disable via SCIM status)

User states: **pending**, **approved**, **rejected**, **email_unverified**, **disabled**.

Note: the **Users** page requires the **admin/sub_admin** role in addition to the **Users** RBAC resource - custom roles cannot open it.

3.2 Roles and permissions

Governance -> **Roles & Permissions** (**/api/roles***, **/api/permissions**, **/api/resources**, **/api/operations**):

- The three **system roles** (**admin**, **sub_admin**, **user**) are seeded automatically and kept in sync on every boot (new permissions are merged in). Customisations you make are preserved, except for **admin**, which always keeps the full catalog.
- Create a **custom role** with a DAC (**all-data** / **own-data**) and a permission set.
- **Resource catalog (33)**: `GuardrailsConfig, GuardrailsProviders, GuardrailRules, UserProvisioning, Cluster, Settings, Users, Logs, Observability, Dashboard, VirtualKeys, ModelProvider, Plugins, MCPGateway, MCPToolGroups, MCPLogs, AdaptiveRouter, AuditLogs, Customers, Teams, RBAC, Governance, RoutingRules, PromptRepository, PromptDeploymentStrategy, SkillsRepository, AccessProfiles, APIKeys, Inference, Metrics, FeatureFlags, CircuitBreaker`
- **Operations (6)**: **Read**, **View**, **Create**, **Update**, **Delete**, **Download** (**View** ≈ **Read**).
- **Scope grants** (**/api/rbac/scope-grants**) limit which records a role can reach.

3.3 Section-scoped access

For a **user**, "Allowed Sections" selects which sidebar areas they can open

(**observability**, **browser-ai**, **models**, **mcp-gateway**, **plugins**, **governance**, **guardrails**,

cluster-config, adaptive-routing, prompt-repository, skills-repository, settings).

Parent grants cover children; **observability/llm-logs** is a child grant; the legacy

observability/browser-ai counts as the top-level **browser-ai**.

3.4 Access profiles

Governance -> **Access Profiles (/api/access-profiles*)**: reusable, versioned policy templates (name, description, tags, spec, calendar-aligned). Activate/deactivate/clone them and apply them to users. Useful for standard personas (contractor, finance, engineering).

3.5 Teams, business units and customers

- **Teams** - group users; assign budgets and rate limits per team.
- **Business Units** - group teams under an org unit with its own budget and rate limits.
- **Customers** - external/tenant grouping for billing and reporting.
- **SCIM / User Provisioning (/api/scim/*)** - connect your IdP (Azure AD/Okta) so joiner/mover/leaver is automatic; SCIM **active=false** maps to user status **disabled**.

4. Virtual keys, budgets and limits

Governance -> **Virtual Keys (/api/governance/virtual-keys*)**:

Task	Endpoint
List keys	GET /api/governance/virtual-keys
Create a key	POST /api/governance/virtual-keys
Rotate keys	POST /api/governance/virtual-keys/rotate
Edit a key	PUT /api/governance/virtual-keys/{id}
Delete	DELETE /api/governance/virtual-keys/{id}
Assign users	/api/governance/virtual-keys/{id}/users
Self-service quota check	GET /api/governance/virtual-keys/quota (key in header)

Practices:

- One key per application or team - never a shared personal key.
- Set a **budget** (cost in USD) and a **rate limit** (requests per minute). Limits are enforced by the governance plugin; Budgets & Limits (Models -> **model-limits**, Providers -> **model-limits**) shows the live values.
- Rotate on staff change or suspected leak; bulk rotation is supported.
- Keys are encrypted at rest when **UNIFAI_ENCRYPTION_KEY** is set.

5. Models, providers and routing

Area	Page / endpoint	What you do
Providers + master keys	Models -> Model Providers (/api/providers* , /api/keys*)	Add provider keys (multiple keys per provider are load-balanced), set network config (timeouts, retries, proxy)
Model catalog	Models -> Model Catalog (/api/models*)	See discovered/live models, enable/disable, inspect pricing
Model settings	Models -> Model Settings (/api/model-configs*)	Per-model parameter defaults and overrides
Pricing	Models -> Pricing Overrides / Model Settings	Override costs so budgets and reports are accurate
Complexity Router	Models -> Complexity Router (/api/governance/complexity-analyzer-config)	Classify requests by complexity and route to cheaper/stronger models
Routing Rules	Models -> Routing Rules (/api/governance/routing-rules, routing-rules/tree)	Conditional routing by model, user, team, key
Circuit Breaker	Models -> Circuit Breaker (/api/circuit-breaker/*)	Fail fast on unhealthy providers/keys; reset policies
Adaptive Routing	Adaptive Routing (/api/load-balancer*)	Load-balancing strategy and per-route weights

Guiding principle: put **least privilege** in front of provider keys. **sub_admin** can see models but cannot read or change master provider keys - only **admin** can.

6. Guardrails vs Guard Rules (do not confuse them)

	Guardrails (gateway)	Guard Rules (Browser AI)
Where	Guardrails -> Rules / Rule Providers	Browser AI -> Guard Rules
Protects	API/inference traffic through the UnifAI gateway	Employee prompts and uploads on public AI websites
Endpoints	/api/guardrails/rules, /api/guardrails/providers	/api/browser-ai/rules*
RBAC resource	GuardrailsConfig, GuardrailsProviders	Logs
Rule types	Provider-based input/output validation	regex or ai_bot (LLM-judged, optional reference image)

Configure both: Guardrails for the governed API path, Guard Rules for the browser path.

7. Browser AI administration (the guard control plane)

Workspace -> **Browser AI**, tabs: Overview, Prompt Logs, Search Logs, Guard Rules, Target Websites, Guard Agents, Guard Insights, Setup.

7.1 Target Websites

GET|POST|PUT|DELETE /api/browser-ai/targets; bulk import via **POST /api/browser-ai/targets/import**.

Field	Meaning
domain	Host to monitor (unique) - e.g. chatgpt.com, api.anthropic.com

Field	Meaning
platform_name	Friendly platform label (ChatGPT, Claude, Gemini, ...)
monitored	Whether the domain is inspected
block_entire_website	Block the whole site instead of analysing prompts
host_role	ui, chat, file or blank (auto) - drives the proxy intercept path
status	MONITORED, PAUSED, BLOCKED

Import-ready samples in the repo: **target_websites_sample.csv / .xlsx** with columns

Domain, Platform Name, Host Role, Monitored, Block Entire Website. Multiple domains per platform are normal - include UI, chat and file hosts.

7.2 Guard Rules

GET|POST|PUT|DELETE /api/browser-ai/rules, POST /rules/import, POST /rules/test-bot, POST /rules/generate-regex.

Field	Values
Rule Name	any text (import key; used with "Overwrite existing")
Rule Type	regex (default) or ai_bot
Pattern	valid regex
Severity	CRITICAL, HIGH, MEDIUM
Action	BLOCK, REDACT, WARN
Warning Message	text shown to the employee when the rule hits
Active	TRUE / FALSE
Description	internal note

Excel/CSV import - accepted files **.xlsx, .xls, .csv**; row 1 must be the header row with the columns above. **ai_bot** rules **cannot be imported**; create those in the UI. Export uses the same column names, so export -> edit -> import is a supported round-trip. Samples:

guard_rules_import_example.csv, guard_rules_sample.csv, guard_rules_enterprise_1000_all_sectors.csv (card PANs, Aadhaar, salary/HR terms, API keys/secrets, private keys, internal codenames). Review them against your own policy before enabling.

AI Guard Bot rules need an LLM: the UI defaults to the **ollama** provider with model **llama3.2** (**/api/browser-ai/ollama-models** lists availability). Vision/reference-image rules accept a reference image up to 512 KB. Always use **test-bot** before enabling.

Rules reach agents within seconds (server cache ~2 s; PAC sync default 3 s).

7.3 Controls

GET|PUT /api/browser-ai/controls - the single-row interaction policy: upload/download warnings and the interaction toggles that apply to every monitored agent. This is where you set the wording

employees see when something is blocked.

7.4 Fleet configuration

GET|PUT /api/browser-ai/fleet-config - company-wide Guard defaults stored in **browser_guard_fleet_config**. Agents pull this on heartbeat, so changes here roll out without rebuilding installers.

7.5 Guard Agents (fleet inventory)

GET /api/browser-ai/agents, filterable by status, source (**endpoint** = Laptop Guard, **network** = server proxy) and search. Each record shows hostname, agent type, last seen, health detail, contact e-mail, uninstall-key state and proxy bundle SHA.

Task	Endpoint
Delete an agent record	DELETE /api/browser-ai/agents/{id} or POST /api/browser-ai/agents/bulk-delete
Set a device contact e-mail	PUT /api/browser-ai/agents/{id}/contact-email
Company uninstall key	GET
Per-agent uninstall key / rotate	GET /api/browser-ai/agents/{id}/uninstall-key , POST .../uninstall-key/rotate
Remote uninstall	POST /api/browser-ai/agents/{id}/remote-uninstall
Agent-initiated uninstall flow	agents/uninstall-verify , agents/uninstall , agents/uninstall-ack , agents/uninstall-status

Statuses: **active**, **uninstall_pending**, **uninstalled**.

Rule of thumb: investigate any **endpoint** agent not seen for more than 7 days - that employee's protection is not reporting.

7.6 Setup and packages

Task	Endpoint
Package info / versions	GET /api/browser-ai/setup/info
Download setup ZIP	GET /api/browser-ai/setup/download.zip (-windows.zip, -mac.zip)
Rebuild & publish Guard code	POST /api/browser-ai/setup/rebuild
Rebuild history	GET /api/browser-ai/setup/rebuild-history
Hot-update bundle info / download	GET /api/browser-ai/setup/proxy-bundle.json , .../proxy-bundle.zip

After changing Guard proxy/agent Python code, run **Rebuild & Publish**; installed agents pick up the bundle automatically (SHA-tracked, with rollback of a marked-bad bundle).

For a full installer release, rebuild packages on the appropriate OS (**make build-guard-windows** / **make build-guard-mac**), copy them into **apps/browser-guard/release/**, and redeploy.

7.7 Prompt Logs and Search Logs (investigation surface)

- **Prompt Logs** (**GET /api/browser-ai/logs**) - filter by platform, status/action, date range and free text (prompt, platform, domain, client IP, rule, agent id/hostname). Statuses: **Allowed**,

Redacted, Warned, Blocked, SiteBlocked, Bot Answered. A row carries the full prompt, the triggered rule, risk/predictive risk and device identity.

- **Search Logs** (**GET** `/api/browser-ai/search-logs`) - engine, browser, incognito flag, query, clicked URL/title, predictive risk.
- **Attachments** - **GET** `/api/browser-ai/attachments/{id}` (privileged).
- **Guard Insights** (**GET** `/api/browser-ai/insights/stats`) - per-agent action totals across the whole database.
- **Warning e-mails** - **POST** `/api/browser-ai/send-warning-email` sends a policy warning to an employee's contact e-mail; sends are logged in **browser_ai_warning_emails**.

Bulk cleanup: **POST** `/api/browser-ai/logs/bulk-delete`, **DELETE** `/api/browser-ai/logs` and the search-log equivalents. Deletions are destructive, admin-only and audited.

7.8 Rollout pattern (recommended)

1. Import Target Websites for the platforms you care about (start with the big 5-10).
2. Import Guard Rules in **WARN/ REDACT** mode first - do not start with blanket **BLOCK**.
3. Install the Guard on a pilot group; watch Prompt Logs and Guard Insights for false positives.
4. Tune regex precision, promote the critical rules to **BLOCK**.
5. Roll out to everyone; set the **uninstall key** and share it only with IT.
6. Add **AI Guard Bot** rules for nuanced cases once regex noise is under control.
7. Review weekly; publish via Rebuild & Publish when you change Guard code.

8. Observability and audit

Area	Page / endpoint	Use it for
LLM Logs	Observability -> LLM Logs (<code>/api/logs*</code>)	Requests, tokens, cost, latency, users, virtual keys, errors
Dashboard	Observability -> Dashboard (<code>/api/logs/dashboard</code>)	Aggregated models/cost/latency/tokens view in one call
Filters / rankings / histograms	<code>/api/logs/filterdata</code> , <code>/api/logs/rankings*</code> , <code>/api/logs/histogram/*</code>	Building reports, spotting outliers
MCP Logs	Observability -> MCP Logs (<code>/api/mcp-logs*</code>)	MCP tool calls: who called what, when, result
Connectors	Observability -> Connectors (<code>/api/connectors</code>)	BigQuery, Kafka, Datadog, New Relic, PubSub sinks
Alert Channels	Observability -> Alert Channels (<code>/api/alert-channels</code>)	Webhook notifications (secrets encrypted at rest)
Audit Logs	Governance -> Audit Logs (<code>/api/audit-logs</code>)	Every mutating <code>/api/*</code> action, captured automatically
Traces	log page trace views	End-to-end request tracing with plugin logs
Prometheus / OTel	telemetry + otel plugins	External monitoring integration

Maintenance actions: **DELETE** `/api/logs` (clear logs), **POST** `/api/logs/recalculate-cost` (recompute costs after a pricing change), **GET** `/api/logs/dropped` (dropped requests). All are admin-only.

9. Platform settings

Settings -> sub-pages and their APIs:

Page	API	Notes
Client Settings	/api/config	Pool size, request body size, logging toggle, content logging, log retention, auth enforcement, allowed origins
Security	/api/config (security section)	Auth/security toggles
Caching	/api/cache	Cache configuration
Compatibility	/api/config (compat)	Parameter conversion between client and provider shapes
Feature Flags	/api/feature-flags	Runtime feature toggles
Logging	/api/config/logging	Which logs are recorded, retention
Observability	/api/connectors	External sinks
Performance Tuning	/api/config (performance)	Read buffer, pool sizing
Pricing Config	/api/pricing*	Default pricing data
Proxy	/api/proxy-config	Outbound proxy for provider calls
API Keys	/api/keys*	Provider keys
Plugins	/api/plugins	Enable/disable and reorder plugins
Cluster	/api/cluster	Multi-node settings (CLUSTER_REPLICATE_SECRET)
MCP Gateway	/api/mcp/settings	Gateway behaviour

GET [/api/config](#) is one of the few API routes reachable with a session in restricted setups - treat it as sensitive and keep it admin-only.

10. MCP Gateway administration

Task	Page / endpoint
Add/manage MCP clients	MCP Gateway -> MCP Catalog (/api/mcp/clients , /api/mcp/client* , /api/mcp/client/{id}/reconnect)
Browse/seed the library	MCP Gateway -> MCP Library (/api/mcp/library , POST /api/mcp/library/force-sync)
Group tools	MCP Gateway -> Tool Groups (/api/mcp/tool-groups*)
OAuth sessions / grants	MCP Gateway -> Auth Sessions (/api/mcp/sessions), OAuth Grants (/api/oauth-grants)
Auth config	MCP Gateway -> MCP Auth Config (/api/mcp/auth-config*)
Tool-call logs	Observability -> MCP Logs

MCP clients are dialled after plugins load; if a client's server is unavailable at boot it is retried by the health monitor. Tool groups and per-user OAuth enforce least privilege on tools.

11. Prompts, skills and prompt deployments

Area	Page / endpoint
Prompt Repository	Prompt Repository (/api/prompt-repo*)
Individual prompts	/api/prompt-repo/prompts
Lifecycle (versions, sessions)	/api/prompt-repo* lifecycle endpoints
Prompt deployments	Prompt Deployments (/api/prompt-deployments)
Skills Repository	Skills (/api/skills* , serving at /api/skills/serve/*)

Use folders and per-user **allowed_prompt_repos** to limit who sees which prompts.

12. Runbooks

12.1 Investigate a possible data leak

1. Browser AI -> **Prompt Logs**: filter by date range and the employee/device (agent hostname or client IP).
2. Look for **Blocked**, **Redacted** or **Warned** rows; open one for the full prompt, the triggered rule and the predictive risk.
3. Check **Guard Insights** for that agent's overall behaviour and **Search Logs** for related queries.
4. If an attachment is involved, download it via [/api/browser-ai/attachments/{id}](#) and follow your evidence-handling process.
5. If warranted, send a policy warning ([/api/browser-ai/send-warning-email](#)) and escalate per your incident policy; note the Audit Log entry for anything you changed.

12.2 Block a newly popular AI tool immediately

1. Browser AI -> **Target Websites** -> add the domain(s) with the right **host_role** (ui/chat/file) and **monitored** set.
2. To hard-stop it, enable **Block Entire Website**; for visibility only, leave it monitored and rely on rules.
3. Allow a few seconds for the PAC/rule refresh, then test in a browser.

12.3 Onboard a department

1. Create a **Team** (and a **Business Unit** if needed) with budget/rate limits.
2. Create or assign an RBAC role (e.g. **user** + Prompt Repository) and an **Access Profile**.
3. Invite users (or let SCIM provision them); approve as needed.
4. Issue a Virtual Key for their application with a budget and rate limit.
5. Send them the Guard installers from Setup (Windows ZIP / macOS ZIP).

12.4 Rotate a leaked Virtual Key

1. Governance -> Virtual Keys -> **Rotate** (**POST** [/api/governance/virtual-keys/rotate](#)).
2. Distribute the new key out-of-band and confirm the old key is rejected.
3. Review LLM Logs for that key's recent usage and report per policy.

12.5 Rotate the Guard fleet secret

1. Generate a new **UNIFAI_GUARD_SECRET** (`openssl rand -base64 32`) and set it in `.env`.
2. Run `python apps/browser-guard/scripts/sync_config_from_env.py` to regenerate Guard config and docs.
3. Rebuild the Guard installers (or Rebuild & Publish) so new/updated agents carry the new secret.
4. Redeploy the backend with **UNIFAI_GUARD_REQUIRE_SECRET=1**.
5. Watch Guard Agents - agents on the old secret will fail to report until updated; plan a maintenance window and communicate it.

12.6 Turn off the Guard on a departing employee's device

1. Browser AI -> **Guard Agents** -> find the device -> **Remote uninstall** (`POST /api/browser-ai/agents/{id}/remote-uninstall`).
2. Confirm the status moves **uninstall_pending** -> **uninstalled**.
3. Disable the user account (Governance -> Users), or let SCIM deprovision it.
4. Rotate any Virtual Keys they had.

12.7 Reduce LLM spend

1. Observability -> **Dashboard / LLM Logs**: identify the biggest spenders (model, user, key, team).
2. Tighten budgets and rate limits on the offending keys/teams.
3. Enable/adjust the **Complexity Router** and **Routing Rules** to send simple work to cheaper models.
4. Verify pricing overrides; run `POST /api/logs/recalculate-cost` after correcting pricing.

12.8 Investigate "a rule isn't firing"

1. Confirm the site is in Target Websites and **monitored**, and the platform's **chat** host is included (many platforms split UI/chat/file hosts).
2. Confirm the rule is **Active**, and check its regex with the **Regex live test** panel in the UI.
3. Confirm the agent is reporting (Guard Agents -> last seen) and the device has no certificate warnings.
4. Remember the propagation delay: server cache ~2 s, PAC sync default 3 s.

13. Routine checklist

Daily

- [] `/health` is green
- [] Guard Agents: any **endpoint** agent not seen recently
- [] Prompt Logs: review notable **Blocked** / **SiteBlocked** events
- [] Backups completed

Weekly

- [] Guard Insights / Search Logs trend review; tune noisy rules
- [] Audit Logs spot check (role changes, key rotations, deletions)
- [] Budget/limit breaches in LLM Logs

- [] Guard rebuild/publish history if code changed

Monthly

- [] Review RBAC roles, access profiles and users (remove stale accounts)
- [] Rotate/verify the uninstall key policy with IT
- [] Pricing accuracy vs provider invoices
- [] Restore-test a backup
- [] Plugin/feature-flag review, log retention and disk check

Quarterly / on change

- [] Rotate **UNIFAI_GUARD_SECRET**, **PASSWORD_RESET_SECRET** and **UNIFAI_ENCRYPTION_KEY** per policy
- [] Re-review Guard Rules against current company policy and regulations
- [] Patch the container image, Postgres and the host

14. Do-nots

- Do not put live secrets in **configs/** or **apps/browser-guard/config/** - they are version-controlled. (**apps/browser-guard/config/unifai_guard_config.json** in this repo currently contains a live-looking **guard_secret**; rotate it and rely on **.env** plus **sync_config_from_env.py**.)
- Do not bind-mount **config.json** as a single file.
- Do not run the laptop Guard and the corporate network proxy on the same browser session.
- Do not hand out **admin** casually - **sub_admin** covers workspace management without master provider keys.
- Do not delete logs casually: it is destructive, audited and destroys evidence.
- Do not treat incognito mode as a privacy control - monitored browsers still report.
- Do not import **ai_bot** rules from Excel/CSV; that is UI-only.

UnifAI - User Guide

Audience: employees and end users (including developers who were issued an API key)

Baseline: repo commit **16d4c3e**

1. What UnifAI is, in plain language

Your company runs two things on one platform:

1. UnifAI Workspace (the dashboard) - a web app where you sign in, keep prompts, and (if you have access) see logs, models, virtual keys and other company AI tooling.

2. UnifAI Guard ("Browser AI") - a small app installed on your work laptop that keeps company data safe while you use public AI websites (ChatGPT, Claude, Gemini, Copilot, DeepSeek, Grok, Perplexity, Poe, Mistral Le Chat, and others your IT team monitors).

In practice:

- You keep using AI websites normally in your browser.
- The Guard runs in the background on your laptop and checks prompts and file uploads against the company's rules before they leave your machine.
- Anything the rules flag is **blocked**, **redacted**, or **warned** - and the event is recorded for your company's security team.

The Guard is a **company security control**, not a personal app. Please do not try to disable or bypass it - IT can see whether it is running.

2. Your account

2.1 Signing up

1. Open the UnifAI URL your IT team gave you (for example **<https://unifai.yourcompany.com>**).
2. Choose **Sign Up**.
3. Enter your details. Depending on company settings, you may receive a **code by e-mail** - enter it to verify your address (**email_unverified** status).
4. Your registration then goes to an administrator for approval (**pending**). You cannot sign in until it is approved.
5. Once approved you can log in. If it is rejected or your account is disabled, login shows the reason sent by the platform.

2.2 Signing in

- Go to the UnifAI URL and log in with your username and password.
- Your session lasts about **24 hours** by default and then you log in again.
- The session cookie is HTTP-only and **Secure** when your company serves the site over HTTPS.

2.3 Forgot password / username

1. On the login screen choose **Forgot password** (or **Forgot username**).
2. A one-time code is e-mailed to you (the code is valid for 10 minutes; there is a short cooldown between requests and a per-hour limit).
3. Enter the code, then set a new password.
4. Too many wrong codes invalidates the request - start again.

2.4 Safety limits you may hit

- Too many failed logins from your network or for your username triggers a temporary lockout with a message telling you how many minutes remain. Wait it out.
- If your account is **disabled** by IT (for example, it was deprovisioned), contact your IT helpdesk.

3. Using the workspace

After login you land in the workspace. Which menu items you see depends on the permissions your administrator gave you. By default the **user** role has read-only access to the **Dashboard**, **LLM/MCP Logs**, **Observability**, **Prompt Repository** and **MCP Gateway**, and your administrator may narrow this down to specific sidebar sections (for example Prompt Repository only).

Common pages you may be granted:

Page	What you can do
Prompt Repository	Save, organise and reuse prompts
Skills Repository	Browse and use shared skills
Dashboard / LLM Logs / MCP Logs	Read-only reporting about AI usage (if granted)
Browser AI	Team-wide view of Guard targets, rules, prompt logs, agents, insights and setup
Virtual Keys / Governance	Manage API credentials, budgets and limits (if granted)
Model Catalog / Providers	See which models are available (read-only)
Playground	Chat with a model through the company gateway
Docs	Links to the full product documentation

Notes:

- If you open a page you were not granted, you are redirected to your first allowed page - you are not shown an error.
- Menu items disappear when you lack permission for that area (RBAC), so the menu is always a reflection of what you can actually do.

3.1 If you have a Virtual Key (API access)

Developers are often issued a **Virtual Key** instead of using the dashboard for AI calls.

- Point your AI client at the company gateway base URL and use the key as the bearer credential:

```
curl https://unifai.yourcompany.com/v1/chat/completions \ -H "Authorization: Bearer <YOUR_VIRTUAL_KEY>" \ -H
"Content-Type: application/json" \ -d
'{"model": "<model-id>", "messages": [{"role": "user", "content": "Hello"}]}'
```

- Existing SDKs can often keep their code and only change the base URL, because the gateway also exposes provider-shaped endpoints such as **/openai/v1/...** and **/anthropic/v1/...**
- Your key may have a **budget** (cost limit) and a **rate limit** (requests per minute). If you hit them you get an error - ask your administrator to adjust the limits.
- Keep your key secret. Do not commit it to code repositories. Report a leaked key immediately so it can be rotated.

4. UnifAI Guard on your laptop

4.1 What it does

- Runs a **local security proxy** on your machine only (**127.0.0.1:18103**) and applies a **PAC** so your browser routes **only the monitored AI websites** through it.
- Checks the text you send and the files you upload against the company's **Guard Rules** before the request reaches the AI site.
- Applies the configured action: **allow**, **redact** (sensitive parts removed), **warn** (a message is shown) or **block** (request stopped).
- Reports the event to the company platform so IT can see what happened.
- Also records **search-engine queries** for monitored browsers (including **incognito/private** windows) and whether a search result was opened.
- Needs no access to the company database and talks only to your company's UnifAI server.

4.2 Installing (Windows)

1. Get **UnifAI_Guard_Windows.zip** from your IT team (or Browser AI -> **Setup** if you have access).
2. **Extract** the ZIP completely (right-click -> *Extract All*). Do not run Setup from inside the ZIP.
3. In the extracted folder run **UnifAI_Guard_Setup.exe**.
4. Keep "**Start automatically at Windows login**" checked.
5. Finish - the Guard starts in the background with no terminal window.
6. Open ChatGPT or another monitored AI site as usual.

4.3 Installing (macOS)

1. Get **UnifAI_Guard_macOS.zip** from IT (or Browser AI -> Setup).
2. Extract it, then double-click **Install_UnifAI_Guard.command**.
3. Approve the macOS prompts (the Guard installs a trusted certificate so it can inspect HTTPS for the monitored sites only).
4. The Guard starts automatically and keeps running in the background.

4.4 Turning the Guard off / uninstalling

This is **gate-controlled** and needs the company **uninstall key** from IT.

- **Windows:** Settings -> Apps -> **UnifAI Guard** -> Uninstall (or Start Menu -> UnifAI Guard ->

Uninstall). Enter the key when prompted.

- **macOS:** double-click **Uninstall_UnifAI_Guard.command** and enter the key.

If your company disabled the key requirement, leaving it blank is allowed - IT will tell you.

IT can also remove the Guard **remotely**; on the server side this shows as

"uninstall pending / uninstalled".

4.5 Where the Guard keeps its files (useful when IT asks)

Platform	Location
Windows data / logs	%LOCALAPPDATA%\UnifAI\Guard\
Windows log file	%LOCALAPPDATA%\UnifAI\Guard\unifai_guard.log
Windows certificate status	%LOCALAPPDATA%\UnifAI\Guard\ca_install_status.txt
macOS data	~/Library/Application Support/UnifAI/Guard/

Keep these files if IT asks for them - they show fastest whether the Guard is connected and whether HTTPS inspection is trusted.

5. What you will actually see

Situation	What happens
You send a normal prompt on a monitored site	Nothing - it is logged as Allowed
Your prompt contains something a rule flags with action BLOCK	The request stops and you see the rule's warning message
Action is REDACT	The sensitive part is removed and the request continues
Action is WARN	You see a warning; the request continues
You visit an AI site marked Block Entire Website	The whole site is blocked
You upload a file that a policy blocks	The upload stops with the configured message
An AI Guard Bot rule handles the prompt	You may see a bot answer instead of the AI site's reply
You search on Google / Bing / DuckDuckGo / etc.	The query and any opened result can be recorded, including in incognito/private mode
The company server is unreachable	Depending on configuration traffic may fail open (allowed) - IT controls this

Data to avoid putting into AI chats: personal IDs (Aadhaar, PAN), card numbers, salaries/HR

information, API keys and secrets, private keys, credentials, and unreleased project names.

*These are typical rule examples and are usually **blocked or redacted**.*

6. Troubleshooting (self-service)

Problem	Try this
"Your connection is not private" / certificate warning on an AI site	Tell IT - the Guard's certificate is not trusted on your machine yet. Report %LOCALAPPDATA%\UnifAI\Guard\ca_install_status.txt (Windows)
The AI site will not load at all	Check whether the site is intentionally blocked by policy; otherwise contact IT
My prompt disappeared / was refused	A Guard rule blocked or redacted it; the warning message names the rule
The Guard seems not to be running	Restart the laptop, then check the log file; ask IT to confirm your device appears in Guard Agents
Login says "waiting for admin approval"	Your registration is pending - an administrator must approve it
Login says "verify your email first"	Enter the code from the Sign Up page
Login says "Too many failed attempts"	Wait the stated number of minutes before trying again
Password reset e-mail does not arrive	Check spam; the request may be rate-limited - wait a couple of minutes and retry. IT must have SMTP configured
I opened a page and got bounced elsewhere	That page is not in your permissions; you were redirected to your first allowed page
An AI client returns 401/403 with my Virtual Key	The key may be revoked/expired, or you exceeded budget/rate limits - ask your administrator
Streaming answers hiccup or arrive at once	Report it to IT (the reverse proxy must not buffer streaming)

Before contacting IT, note: what you were doing, the site, the exact message, the time (with timezone), and your device name. That lets them find the matching Prompt Log entry quickly.

7. Privacy and expectations (straight answers)

Question	Answer
Can IT see the prompts I send to AI sites?	Yes, for monitored websites. Prompt text, site, domain, device and the verdict are stored so the security team can investigate incidents
Can IT see my personal browsing?	The Guard proxies only the AI websites configured as targets, so unrelated browsing is not inspected. Search-engine queries and clicked results are recorded for monitored browsers
Does incognito/private mode hide my activity?	No - for monitored browsers the Guard records the query and marks it as incognito/private
Are uploaded files inspected?	Yes. File type, name and size are recorded, and file bytes are stored on the server when policy requires review
Does the Guard send my data anywhere else?	No - only to your company's UnifAI server. Your requests still go to the AI website you are using
Can I turn the Guard off?	Only with the company uninstall key, or if IT removes it remotely. It is a company control on a company device
How long is data kept?	Company policy - up to the configured retention (default 365 days for logs)
Can I ask for my data to be deleted?	Contact your IT/security team; only administrators can delete logs

8. Quick-reference card (share with colleagues)

Sign in → <https://<unifai-url>> (ask IT for the key/URL) Forgot password → Login page → Forgot password → e-mail code (valid 10 min) Guard install (Win)→ Extract UnifAI_Guard_Windows.zip → UnifAI_Guard_Setup.exe Guard install (Mac)→ Extract UnifAI_Guard_macOS.zip → Install_UnifAI_Guard.command Guard off → Needs the company uninstall key from IT Guard log (Win) → %LOCALAPPDATA%\UnifAI\Guard\unifai_guard.log Cert status (Win) → %LOCALAPPDATA%\UnifAI\Guard\ca_install_status.txt Report a problem → Site + message + time + device name Never put in AI → IDs, card numbers, salary/HR data, API keys, credentials, secret projects

UnifAI / Raksha Enterprise Database Configuration & Persistence Guide

Audience: Database Administrators (DBA), DevOps Engineers, Platform Architects

Scope: PostgreSQL architecture, installation, user/database provisioning, **pg_hba.conf** security, SSL/TLS encryption, GORM schema management, table reference, PgBouncer pooling, **postgresql.conf** performance tuning, automated backup scripts, and disaster recovery.

Baseline: Repo commit **16d4c3e**

1. Database Architecture & Engine Support

UnifAI uses GORM as its persistence abstraction layer. The platform separates state into two logical stores:

- 1. **ConfigStore (framework/configstore):** Core configuration, identity, RBAC, access profiles, model providers, virtual keys, routing rules, prompts, skills, and settings.
- 2. **LogsStore (framework/logstore):** LLM inference logs, MCP tool execution traces, Browser AI employee prompt audit logs, search engine logs, and materialized aggregations.

Both stores can reside on the same PostgreSQL database or be split across dedicated database instances in high-scale environments.

Supported Database Engines

Engine	Production Recommended	Minimum Version	Notes
PostgreSQL	YES (Enterprise Standard)	14.x / 15.x / 16.x	Full JSONB support, GIN indexing, robust concurrency, streaming replication.
MySQL / MariaDB	Supported	8.0+ / 10.6+	Supported via framework/mysqlconn .
SQLite	NO (Development Only)	3.40+	Built via CGO go-sqlite3 (sqlite_static) . For single-developer local testing only.

2. PostgreSQL 16 Installation on Ubuntu / Debian

Execute as **root** or an administrative user with **sudo**:

```
# 1. Add official PostgreSQL APT repository
sudo apt update && sudo apt install -y curl ca-certificates gnupg
lsb-release sudo install -d /etc/apt/keyrings curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc
| sudo gpg --dearmor -o /etc/apt/keyrings/postgresql.gpg echo "deb
[signed-by=/etc/apt/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg
main" | sudo tee /etc/apt/sources.list.d/pgdg.list
# 2. Install PostgreSQL 16 server, client, and contrib
extensions sudo apt update sudo apt install -y postgresql-16 postgresql-contrib-16
# 3. Enable and verify
service status sudo systemctl enable postgresql sudo systemctl status postgresql
```

3. Database & User Provisioning (Exact Project Schema)

Connect to the PostgreSQL console as the **postgres** superuser:

```
sudo -u postgres psql
```

Execute the exact provisioning SQL commands matching project **.env.example**:

```
-- 1. Create dedicated application user with SCRAM-SHA-256 encryption CREATE USER agent_unify WITH ENCRYPTED
PASSWORD 'change-me-strong-password'; -- 2. Create production database with UTF-8 encoding CREATE DATABASE
raksha_new WITH OWNER agent_unify ENCODING 'UTF8' LC_COLLATE 'en_US.UTF-8' LC_CTYPE 'en_US.UTF-8'; -- 3.
Connect to the newly created database \c raksha_new -- 4. Grant schema and default privileges GRANT ALL
PRIVILEGES ON DATABASE raksha_new TO agent_unify; GRANT ALL ON SCHEMA public TO agent_unify; ALTER DEFAULT
PRIVILEGES IN SCHEMA public GRANT ALL ON TABLES TO agent_unify; ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT ALL ON SEQUENCES TO agent_unify; ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON FUNCTIONS TO
agent_unify; -- 5. Install extensions required for UUIDs, full-text, and trigram search CREATE EXTENSION IF
NOT EXISTS "uuid-ossf"; CREATE EXTENSION IF NOT EXISTS "pg_trgm"; CREATE EXTENSION IF NOT EXISTS
"btree_gist"; -- Verify installed extensions \dx -- Exit psql \q
```

4. Network Security & **pg_hba.conf** Configuration

Edit **/etc/postgresql/16/main/pg_hba.conf** to enforce **scram-sha-256** password hashing and restrict incoming connections strictly to authorized hosts:

```
# TYPE DATABASE USER ADDRESS METHOD # Local administrative access via UNIX domain socket local all postgres
peer local all all scram-sha-256 # Localhost IPv4 loopback (when UnifAI runs on the same server) host
raksha_new agent_unify 127.0.0.1/32 scram-sha-256 # Localhost IPv6 loopback host raksha_new agent_unify
::1/128 scram-sha-256 # Docker Container Bridge Network (e.g. 172.16.0.0/12 or 10.0.0.0/8) host raksha_new
agent_unify 172.16.0.0/12 scram-sha-256 host raksha_new agent_unify 10.0.0.0/8 scram-sha-256 # Reject all
other unauthorized external traffic host all all 0.0.0.0/0 reject
```

Listen Address in **postgresql.conf**

Edit **/etc/postgresql/16/main/postgresql.conf**:

```
listen_addresses = 'localhost, 172.16.0.1' # Replace with your VPC private IP port = 5432 max_connections =
300
```

Reload PostgreSQL:

```
sudo systemctl reload postgresql
```

5. SSL/TLS Database Encryption (In-Transit Protection)

Enterprise production environments must encrypt all database traffic across the network.

5.1 Generating PostgreSQL SSL Certificates

```
sudo mkdir -p /etc/postgresql/ssl && cd /etc/postgresql/ssl # Generate 4096-bit private key sudo openssl
genrsa -out server.key 4096 sudo chmod 600 server.key # Generate self-signed certificate (or use corporate
CA) sudo openssl req -new -x509 -days 3650 -key server.key -out server.crt \ -subj
"/CN=db.internal.yourcompany.com" sudo chmod 644 server.crt sudo chown -R postgres:postgres
/etc/postgresql/ssl
```

5.2 Enabling SSL in postgresql.conf

```
ssl = on ssl_cert_file = '/etc/postgresql/ssl/server.crt' ssl_key_file = '/etc/postgresql/ssl/server.key'
ssl_min_protocol_version = 'TLSv1.2' ssl_ciphers = 'HIGH:!aNULL:!MD5'
```

Restart PostgreSQL:

```
sudo systemctl restart postgresql
```

5.3 Configuring UnifAI .env for SSL Mode

In your **.env** file, configure:

```
DB_TYPE=postgres DB_HOST=127.0.0.1 DB_PORT=5432 DB_NAME=raksha_new DB_USER=agent_unify
DB_PASSWORD=change-me-strong-password DB_SSL_MODE=require
```

Available SSL modes in GORM / pq driver:

- **disable**: Plaintext communication (development only).
- **require**: TLS encryption enforced; server certificate is not verified against CA.
- **verify-ca**: TLS encryption enforced; server certificate verified against trusted CA.
- **verify-full**: TLS encryption enforced; verifies certificate CA and validates server hostname.

6. Schema & Table Reference

UnifAI automatically runs GORM auto-migrations on startup (**framework/configstore/tables/migrations** and **framework/logstore/migrations.go**).

6.1 ConfigStore Tables (framework/configstore/tables/)

Table Name	Purpose	Key Columns / Schema Details
governance_users	User accounts & identity	id, username(uniq), email, password, role(admin/sub_admin/user), status(pending/approved/rejected/email_unverified/disabled), budget, rate_limit, allowed_prompt_repos, allowed_sections, reviewed_at, created_at
rbac_roles	RBAC role definitions	id, name(uniq), description, is_system_role, dac(all-data/own-data), permission_ids(JSON), created_at
access_pro_files	Policy templates	id, name(uniq), description, is_active, version, calendar_aligned, tags(JSON), spec(JSON), created_at
governance_business_units	Organizational units	id, name(uniq), team_ids(JSON), budget(JSON), rate_limit(JSON)
virtual_keys	Gateway API credentials	id, key_name, hashed_key, prefix, team_id, customer_id, budget, rate_limit, allowed_models(JSON), created_at

Table Name	Purpose	Key Columns / Schema Details
model_providers	AI provider credentials	id, name, provider_type, encrypted_api_key, base_url, weight, priority, is_active
models	Model catalog entries	id, model_name, provider_id, pricing_id, context_window, is_enabled
routing_rules	Dynamic model router	id, name, priority, condition_tree(JSON), target_model, is_active
circuit_breaker_policies	Upstream failure shields	id, name, failure_threshold, recovery_timeout_sec, half_open_requests
system_settings	Platform configuration	id, company_name, company_logo, allowed_origins, session_ttl_sec
smtp_configs	Email notification settings	id, host, port, username, encrypted_password, from_email, use_tls

6.2 Browser AI Tables (framework/logstore/browser_ai.go)

Table Name	Purpose	Key Columns / Schema Details
browser_ai_logs	Intercepted prompt audit trail	id, timestamp, platform, domain, user_prompt_full, action(ALLOW/BLOCK/REDACT/WARN), status, rule_triggered, risk_score, predictive_risk, agent_id, agent_hostname, client_ip, reply_bot_text, attachment_name, attachment_size, attachment_path, metadata(JSON)
browser_ai_search_logs	Search query telemetry	id, timestamp, engine, browser, is_incognito, query, clicked_url, clicked_title, agent_id, predictive_risk
browser_guard_rules	Guard DLP detection rules	id, name, pattern(regex), rule_type(regex/ai_bot), severity(CRITICAL/HIGH/MEDIUM), action(BLOCK/REDACT/WARN), warning_message, active, bot_provider, bot_model, bot_prompt
browser_ai_target_websites	Monitored AI platforms	id, domain(uniq), platform_name, monitored, block_entire_website, host_role(ui/chat/file), status(MONITORED/PAUSED/BLOCKED)
browser_ai_controls_settings	Global interaction policy	id, block_title, block_message, file_upload_warning, interaction_toggles(JSON)
browser_ai_agents	Fleet endpoint inventory	id, hostname, agent_type(endpoint/network), status(active/uninstall_pending/uninstalled), last_seen_at, health_detail, contact_email, uninstall_key_hash, uninstall_key_enc, proxy_bundle_sha
browser_guard_fleet_config	Agent heartbeat defaults	id, pac_sync_interval, fail_open, eval_timeout_sec, created_at
browser_guard_rebuilt_logs	Guard packaging logs	id, status(success/failed), bundle_sha, message, log, created_at

7. Connection Pooling with PgBouncer

In high-concurrency environments (over 500 active Browser Guard agents or 100k+ daily inference calls), use **PgBouncer** to pool database connections.

7.1 Installation

```
sudo apt install -y pgbouncer
```

7.2 Configuration (/etc/pgbouncer/pgbouncer.ini)

```
[databases] raksha_new = host=127.0.0.1 port=5432 dbname=raksha_new [pgbouncer] logfile =  
/var/log/postgresql/pgbouncer.log pidfile = /var/run/postgresql/pgbouncer.pid listen_addr = 127.0.0.1  
listen_port = 6432 auth_type = scram-sha-256 auth_file = /etc/pgbouncer/userlist.txt pool_mode = transaction  
max_client_conn = 1500 default_pool_size = 50 min_pool_size = 10 reserve_pool_size = 5 reserve_pool_timeout =  
5 server_idle_timeout = 300
```

Generate **/etc/pgbouncer/userlist.txt**:

```
"agent_unify" "SCRAM-SHA-256$..."
```

Enable and start PgBouncer:

```
sudo systemctl enable pgbouncer sudo systemctl restart pgbouncer
```

Point UnifAI **.env** to PgBouncer port:

```
DB_PORT=6432
```

8. Production PostgreSQL Performance Tuning (postgresql.conf)

Tune these parameters based on your database host RAM:

Recommended Settings for 16 GB RAM Server

```
# Memory Configuration shared_buffers = 4GB # 25% of total RAM effective_cache_size = 12GB # 75% of total RAM  
maintenance_work_mem = 1GB work_mem = 16MB # Per-query sort memory wal_buffers = 16MB # Checkpoints and Write  
Ahead Log (WAL) min_wal_size = 2GB max_wal_size = 16GB checkpoint_completion_target = 0.9 checkpoint_timeout  
= 15min # Disk I/O & SSD / NVMe Optimization random_page_cost = 1.1 # Fast SSD storage  
effective_io_concurrency = 200 # Concurrent asynchronous disk operations # Worker Processes  
max_worker_processes = 8 max_parallel_workers_per_gather = 4 max_parallel_workers = 8  
max_parallel_maintenance_workers = 4
```

9. Automated Daily Backup Script & Disaster Recovery

9.1 On-Demand Backup Command

```
# Custom compressed format backup (Recommended) pg_dump -h 127.0.0.1 -p 5432 -U agent_unify -Fc -b -v -f  
"raksha_new_$(date +%Y%m%d_%H%M%S).dump" raksha_new
```

9.2 Automated Daily Backup Script (/opt/unifai/scripts/db_backup.sh)

```
#!/bin/bash # ===== # UnifAI /
Raksha PostgreSQL Production Daily Backup Script #
===== set -e
BACKUP_DIR="/var/backups/unifai/db" DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/raksha_new_${DATE}.dump" LOG_FILE="/var/log/unifai_db_backup.log"
RETENTION_DAYS=30 mkdir -p "${BACKUP_DIR}" echo "[$(date)] Starting UnifAI PostgreSQL backup..." >>
"${LOG_FILE}" export PGPASSWORD="change-me-strong-password" pg_dump -h 127.0.0.1 -p 5432 -U agent_unify -Fc
-b -v raksha_new > "${BACKUP_FILE}" 2>> "${LOG_FILE}" if [ $? -eq 0 ]; then SIZE=$(du -h "${BACKUP_FILE}" |
cut -f1) echo "[$(date)] Backup completed successfully: ${BACKUP_FILE} (${SIZE})" >> "${LOG_FILE}" else echo
"[$(date)] ERROR: Backup failed!" >> "${LOG_FILE}" exit 1 fi # Retention policy: remove backups older than 30
days find "${BACKUP_DIR}" -name "raksha_new_*.dump" -type f -mtime +${RETENTION_DAYS} -delete echo
"[$(date)] Cleaned up backups older than ${RETENTION_DAYS} days." >> "${LOG_FILE}"
```

Make executable and schedule in Crontab:

```
sudo chmod +x /opt/unifai/scripts/db_backup.sh # Add to root crontab (sudo crontab -e): 0 2 * * *
/opt/unifai/scripts/db_backup.sh >/dev/null 2>&1
```

9.3 Step-by-Step Disaster Recovery Restore Procedure

In the event of database corruption or hardware failover:

```
# 1. Terminate all active application connections to the database sudo -u postgres psql -c "SELECT
pg_terminate_backend(pid) FROM pg_stat_activity WHERE datname = 'raksha_new' AND pid <> pg_backend_pid();" #
2. Drop and recreate database sudo -u postgres psql -c "DROP DATABASE IF EXISTS raksha_new;" sudo -u postgres
psql -c "CREATE DATABASE raksha_new WITH OWNER agent_unify ENCODING 'UTF8';" # 3. Restore database schema,
tables, and data using pg_restore sudo -u postgres pg_restore -d raksha_new -v -0 -x
"/var/backups/unifai/db/raksha_new_20261003_120000.dump" # 4. Re-grant privileges to application user sudo
-u postgres psql -d raksha_new -c "GRANT ALL ON ALL TABLES IN SCHEMA public TO agent_unify; GRANT ALL ON ALL
SEQUENCES IN SCHEMA public TO agent_unify;" # 5. Verify restored tables and count sudo -u postgres psql -d
raksha_new -c "\dt"
```

End of Database Configuration Guide.

UnifAI / Raksha Enterprise Docker & SSL/TLS Configuration Guide

Audience: DevOps Engineers, Cloud Engineers, SREs, Systems & Security Administrators

Scope: Docker topology, multi-container orchestration, exhaustive Docker command handbook, SSL/TLS certificate acquisition via Certbot, OpenSSL self-signed generation, Browser Guard Root CA generation, OS trust store deployment commands, Nginx reverse proxy integration, and diagnostics.

Baseline: Repo commit **16d4c3e**

1. Container Topology & Architecture

UnifAI deploys as a multi-container Docker Compose application:

[illegible]

Services Defined in `docker-compose.yml`

1. raksha_tech (Primary Application):

- Multi-stage build (**deploy/docker/Dockerfile.local**): **node:25-alpine** builds React UI \rightrightarrow **golang:1.26.4-alpine** builds Go binary \rightrightarrow **alpine:3.23** lightweight runtime.
- Serves the UI, **/v1/*** inference gateway, **/api/*** management plane, and Browser AI endpoints.

2. raksha_browser_ai_proxy (Optional Network Proxy):

- Enabled via profile: **docker compose --profile network-proxy up -d**.
- Runs **mitmproxy/mitmproxy:latest** with the Browser Guard Python addon for office/lab PAC proxying.

2. Complete Docker & Docker Compose Command Handbook

All commands should be executed from the project root (**d:\unifai_project** or **/opt/unifai**).

2.1 Network Setup (Prerequisite)

The compose file references an external network for Ollama (**1panel-network**). Create it once:

```
# Create external Ollama network (required before startup) docker network create 1panel-network # Create
primary application bridge network (if custom name specified in .env) docker network create raksha-network #
List all Docker networks docker network ls
```

2.2 Container Startup & Rebuild Commands

```
# 1. Start primary UnifAI container in background (detached mode) docker compose up -d # 2. Start both
primary backend AND the optional network proxy docker compose --profile network-proxy up -d # 3. Start with
fresh rebuild (forces re-compilation of UI and Go binary) docker compose up -d --build # 4. Clean rebuild
without using any Docker build cache docker compose build --no-cache && docker compose up -d # 5. Start in
foreground (prints logs directly to console; Ctrl+C to stop) docker compose up
```

2.3 Status, Monitoring & Inspection Commands

```
# View status of running containers, ports, and healthcheck status docker compose ps # View status of all
containers including stopped or exited ones docker compose ps -a # View real-time CPU %, Memory usage,
Network I/O, and PIDs docker stats raksha_tech # Inspect detailed container JSON metadata (IP address,
mounts, environment) docker inspect raksha_tech # Inspect network to see assigned IP addresses of connected
containers docker network inspect raksha-network docker network inspect 1panel-network
```

2.4 Live Logs & Diagnostics Commands

```
# Follow real-time logs of all containers docker compose logs -f # Follow logs of primary Go backend with
last 100 lines docker compose logs -f --tail=100 raksha_tech # Follow logs of network proxy container docker
compose logs -f --tail=100 raksha_browser_ai_proxy # Filter logs for errors or warnings docker compose logs
raksha_tech | grep -i "error" # Check startup banner and plugin status table docker compose logs raksha_tech
| grep -E "successfully started|plugin status:"
```

2.5 Container Execution & In-Container Debugging

```
# Open an interactive shell inside the running backend container docker exec -it raksha_tech /bin/sh # Test
backend healthcheck directly from inside the container docker exec -it raksha_tech curl -v
http://localhost:6000/health # Verify environment variables loaded inside the container docker exec -it
raksha_tech env | grep -E "APP_PORT|DB_HOST|SERVER_DOMAIN" # Inspect data directory contents inside container
docker exec -it raksha_tech ls -la /app/data
```

2.6 Stopping, Restarting & Cleanup Commands

```
# Restart the backend container gracefully (after modifying .env) docker compose restart raksha_tech # Stop
running containers without deleting them docker compose stop # Stop and remove containers, networks, and
internal links docker compose down # Stop and remove containers, networks, and anonymous volumes (bind mounts
preserved) docker compose down -v # Remove dangling unused images docker image prune -f # Clean up stopped
containers, unused networks, and dangling images docker system prune -f # Full nuclear prune: removes all
unused images, stopped containers, and unused volumes (CAUTION) docker system prune -a --volumes -f
```

3. SSL/TLS Certificate Generation & Management

Production enterprise deployments require valid TLS certificates. This section provides exact commands for obtaining certificates via Let's Encrypt (Certbot), generating private self-signed certificates with OpenSSL, and generating the Root CA certificate for Browser Guard interception.

3.1 Obtaining Free Public SSL Certificates with Let's Encrypt (Certbot)

Certbot provides trusted certificates accepted by all modern operating systems and web browsers.

Prerequisites

- Your domain name (e.g. **unifai.yourcompany.com**) must have an active DNS **A** record pointing to your server's public IP address.
- Ports **80** and **443** must be open on your server firewall.

Method A: Standalone Mode (Before starting Nginx)

```
# Install Certbot sudo apt update && sudo apt install -y certbot # Request certificate in standalone mode
sudo certbot certonly --standalone \ --preferred-challenges http \ -d unifai.yourcompany.com \ --email
security-admin@yourcompany.com \ --agree-tos \ --no-eff-email # Certificates are saved to: # Certificate &
Full Chain: /etc/letsencrypt/live/unifai.yourcompany.com/fullchain.pem # Private Key:
/etc/letsencrypt/live/unifai.yourcompany.com/privkey.pem
```

Method B: Webroot Mode (With Nginx already running)

```
# Create certbot challenge directory sudo mkdir -p /var/www/certbot # Request certificate using webroot sudo
certbot certonly --webroot \ -w /var/www/certbot \ -d unifai.yourcompany.com \ --email
security-admin@yourcompany.com \ --agree-tos # Test automated certificate renewal sudo certbot renew
--dry-run
```

Method C: Automated Renewal Crontab

Certbot certificates expire after 90 days. Set up automatic renewal:

```
# Add cron job to renew twice daily and reload Nginx echo "0 3,15 * * * root certbot renew --quiet
--post-hook 'systemctl reload nginx'" | sudo tee -a /etc/crontab
```

3.2 Generating Self-Signed SSL/TLS Certificates with OpenSSL

For internal labs, air-gapped VPCs, or testing environments without public DNS:

Single-Command 4096-bit RSA Certificate with SAN (Subject Alternative Names)

```
# Create directory for custom certificates sudo mkdir -p /etc/ssl/unifai && cd /etc/ssl/unifai # Generate
private key and self-signed certificate valid for 3 years (1095 days) sudo openssl req -x509 -nodes -days
1095 -newkey rsa:4096 \ -keyout unifai_server.key \ -out unifai_server.crt \ -subj
"/C=US/ST=California/L=SanFrancisco/O=YourCompany/OU=IT/CN=unifai.yourcompany.com" \ -addext
"subjectAltName=DNS:unifai.yourcompany.com,DNS:localhost,IP:127.0.0.1" # Restrict private key permissions
sudo chmod 600 unifai_server.key sudo chmod 644 unifai_server.crt
```

Inspecting and Verifying the Generated Certificate

```
# Print certificate human-readable details, validity dates, and SAN extensions openssl x509 -in
unifai_server.crt -text -noout # Verify certificate matches private key modulus openssl x509 -noout -modulus
-in unifai_server.crt | openssl md5 openssl rsa -noout -modulus -in unifai_server.key | openssl md5 # Output
MD5 hashes MUST be identical!
```

3.3 Generating Root CA Certificate for Raksha Browser Guard (MITM Proxy)

To inspect HTTPS traffic between employee browsers and target Generative AI websites (ChatGPT, Claude, Gemini), Browser Guard requires an internal Root Certificate Authority (CA).

Step-by-Step Root CA Creation with OpenSSL

```
# Create CA authority directory mkdir -p /opt/unifai/certs/ca && cd /opt/unifai/certs/ca # 1. Generate Root CA Private Key (4096-bit RSA) openssl genrsa -out raksha-ca.key 4096 chmod 400 raksha-ca.key # 2. Generate Root CA Certificate (valid for 10 years / 3650 days) openssl req -x509 -new -nodes -key raksha-ca.key -sha256 -days 3650 \ -out raksha-ca.crt \ -subj "/C=US/ST=State/L=City/O=Raksha Security Enterprise/OU=Fleet Security/CN=Raksha Root CA - Enterprise AI Guard" # 3. Create combined PEM file for mitmproxy cat raksha-ca.key raksha-ca.crt > mitmproxy-ca.pem chmod 600 mitmproxy-ca.pem
```

3.4 Installing the Root CA Certificate into Client Operating Systems

For Browser Guard to inspect HTTPS sessions without browser security warnings (**ERR_CERT_AUTHORITY_INVALID**), the Root CA certificate (**raksha-ca.crt**) must be installed into the client trust store.

A. Windows Client Installation (Automated via CMD / PowerShell / Inno Setup)

Execute as **Administrator**:

```
:: Using built-in Windows certutil utility certutil -addstore -f "Root" "C:\path\to\raksha-ca.crt"
```

Verify installation in Windows Certificate Manager:

```
certutil -verifystore "Root" "Raksha Root CA - Enterprise AI Guard"
```

B. macOS Client Installation (Terminal / Jamf / Kandji)

Execute with **sudo**:

```
# Add certificate to macOS System Keychain and grant full trust for SSL/TLS sudo security add-trusted-cert \ -d \ -r trustRoot \ -k /Library/Keychains/System.keychain \ /path/to/raksha-ca.crt
```

Verify on macOS:

```
security find-certificate -c "Raksha Root CA - Enterprise AI Guard" /Library/Keychains/System.keychain
```

C. Linux (Ubuntu / Debian) Client Installation

```
# Copy certificate to trusted store directory sudo cp raksha-ca.crt /usr/local/share/ca-certificates/raksha-ca.crt # Update system certificate authorities sudo update-ca-certificates
```

D. Mozilla Firefox NSS Database Installation (Cross-Platform)

Firefox uses its own internal NSS certificate database rather than the OS trust store. To install into Firefox profiles via CLI:

```
# Install libnss3-tools sudo apt install -y libnss3-tools # Linux # or choco install nss -y # Windows # Import into all Firefox user profiles for certDB in $(find $HOME/.mozilla/firefox* -name "cert9.db"); do certdir=$(dirname ${certDB}) certutil -A -n "Raksha Root CA" -t "TCu,Cu,Tu" -i /path/to/raksha-ca.crt -d sql:${certdir} done
```

4. Reverse Proxy Integration: Connecting Nginx to Docker

When deploying via Docker Compose, Nginx runs on the host server (or in a dedicated edge container) and proxies incoming HTTPS requests to the Docker container port (**6000**).

4.1 Production Nginx Configuration File

Save to **/etc/nginx/sites-available/unifai:**

```
# Upstream pointing to Docker published port on localhost upstream docker_unifai_backend { server
127.0.0.1:6000; keepalive 32; } # Plain HTTP to HTTPS redirect server { listen 80; listen [::]:80;
server_name unifai.yourcompany.com; return 301 https://$host$request_uri; } # Production HTTPS Server server
{ listen 443 ssl http2; listen [::]:443 ssl http2; server_name unifai.yourcompany.com; # SSL Certificates
ssl_certificate /etc/letsencrypt/live/unifai.yourcompany.com/fullchain.pem; ssl_certificate_key
/etc/letsencrypt/live/unifai.yourcompany.com/privkey.pem; # Protocols and Ciphers ssl_protocols TLSv1.2
TLSv1.3; ssl_ciphers HIGH:!aNULL:!MD5; ssl_prefer_server_ciphers on; ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d; # Max upload limit for Guard installers and document embeddings client_max_body_size
100M; # Proxy to Docker container location / { proxy_pass http://docker_unifai_backend; proxy_http_version
1.1; # WebSocket support proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade"; #
Request forwarding headers proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto https;
proxy_set_header X-Forwarded-Host $host; proxy_set_header X-Forwarded-Port 443; # Disable buffering for
real-time streaming AI responses (SSE) proxy_buffering off; proxy_read_timeout 600s; proxy_connect_timeout
60s; } }
```

Enable and reload Nginx:

```
sudo ln -sf /etc/nginx/sites-available/unifai /etc/nginx/sites-enabled/ sudo nginx -t && sudo systemctl
reload nginx
```

5. SSL/TLS Verification & Diagnostics Commands

Always test your SSL configuration and verify end-to-end handshake:

```
# 1. Test TLS Handshake and inspect presented certificate chain openssl s_client -connect
unifai.yourcompany.com:443 -servername unifai.yourcompany.com # 2. Check HTTP status, redirection, and
headers using curl curl -vI https://unifai.yourcompany.com/health # 3. Verify TLS 1.3 protocol negotiation
openssl s_client -connect unifai.yourcompany.com:443 -tls1_3 # 4. Check certificate expiration date directly
from remote server echo | openssl s_client -connect unifai.yourcompany.com:443 -servername
unifai.yourcompany.com 2>/dev/null | openssl x509 -noout -dates
```

End of Docker & SSL/TLS Configuration Guide.

Raksha

R.A.K.S.H.A - Real-time AI Knowledge Security & Hazard Assessment

Operating Enterprise: YesPanchi Group of Companies

Document Title: Enterprise Technical Architecture, Port Specifications & Operations Manual

Document Code: RAKSHA-TECH-SPEC-2026-V2.4

Classification: Enterprise Confidential - Internal Engineering, DevOps & SecOps

Version Baseline: 2.4.0 (Enterprise Production Release)

Publication Date: October 2026

Executive Platform Overview

RAKSHA (Real-time AI Knowledge Security & Hazard Assessment) is an enterprise cybersecurity and AI governance platform developed by YesPanchi Group of Companies:

- **Centralized API Gateway:** High-performance proxy built on Go 1.26 **fasthttp** mediating enterprise microservices to downstream LLM providers (OpenAI, Anthropic, Bedrock, Ollama) with prompt guardrails, PII redaction, rate limiting, and virtual key budgeting.
- **Endpoint Browser Guard:** Workstation agent and local MitM proxy built on Python 3.11 **mitmproxy** that intercepts employee browser traffic to public Generative AI websites (ChatGPT, Claude, Gemini) to prevent data loss and unauthorized uploads.

Metadata Field	Platform Specification Details
Platform Name	Raksha (Real-time AI Knowledge Security & Hazard Assessment)
Enterprise Parent	YesPanchi Group of Companies
Dual-Plane Architecture	Go 1.26 Central API Gateway + Python 3.11 Endpoint MitM Interceptor
Primary Database	PostgreSQL 16 Enterprise (SCRAM-SHA-256 Authentication)
Deployment Engine	Multi-Container Docker Compose on Alpine Linux 3.23.4 Hardened Base
Supported Workstations	Microsoft Windows 10/11 (x64), Apple macOS (Apple Silicon / Intel), Linux

Table of Contents

Section No.	Chapter Title	Topic Scope & Subject Matter	Target Page
Chapter 1	Programming Languages & Version Matrix	Pinned versions for Go, React, TypeScript, Python, Alpine, and PostgreSQL	Page 3
Chapter 2	Network Architecture & Port Matrix	Complete port assignments (6000, 3100, 18182, 18103, 18195, 5432, 11434, 443)	Page 4
Chapter 3	Docker Installation & Command Handbook	1-line installation scripts, network setup, and daily container operations CLI	Page 5
Chapter 4	Cryptographic Secrets & Certificates Handbook	OpenSSL generation for .env secrets, Certbot HTTPS, and Root CA deployment	Page 6
Chapter 5	Production Deployment & Verification Matrix	4-step launch checklist, health check verification, and troubleshooting guide	Page 7

1. Programming Languages & Version Matrix

Every component of the Raksha platform is pinned to deterministic production versions to ensure reproducible builds, enterprise stability, and zero runtime drift:

Component / Layer	Technology	Exact Version	Source File in Repo	Technical Purpose & Function
Backend Core	Go (Golang)	1.26.4	go.work, core/go.mod	High-throughput gateway, proxying, and RBAC
HTTP Engine	valyala/fasthttp	v1.69.0	transports/raksha-http	Low-latency, memory-optimized HTTP engine
Database ORM	GORM	v1.31.1	framework/configstore	Schema migrations & PostgreSQL/SQLite driver
Embedded Database	mattn/go-sqlite3	v1.14.33	CGO Static Link	Fast local caching and offline fallback storage
Frontend Framework	React	19.2.3	ui/package.json	Modern component-based administration console
Frontend Language	TypeScript	5.9.2	ui/package.json	Strict type safety across all dashboard components
UI Build System	Vite	8.0.16	ui/package.json	High-speed frontend bundler and development server
Styling Engine	Tailwind CSS	4.1.12	ui/package.json	Responsive utility-first design system
Desktop Agent	Python	3.11+ / 3.12	apps/browser-guard	Workstation background agent & local MitM proxy
Interception Engine	mitmproxy	>= 10.0.0	apps/browser-guard	Real-time GenAI HTTP/HTTPS traffic analysis & DLP
Binary Compiler	PyInstaller	>= 6.0.0	apps/browser-guard	Compiles Python agent into standalone executable
Windows Installer	Inno Setup	6.2+	apps/browser-guard	Generates enterprise Windows silent setup installer
Container Base OS	Alpine Linux	3.23.4	Dockerfile.local	Minimal, security-hardened container runtime
Primary Database	PostgreSQL	16.x	docker-compose.yml	Production database for config, keys, and audit logs
Local AI Engine	Ollama	v0.3+ / v0.5+	docker-compose.yml	Offline local LLM model execution (Port 11434)

Key Architecture Principles: - Go fasthttp provides microsecond-level proxying latency for high-concurrency API inference. - Python mitmproxy delivers deep packet inspection and prompt redaction directly on employee endpoints. - Alpine Linux keeps the production container attack surface minimal (< 85 MB base).

2. Network Architecture & Port Matrix

Raksha requires specific network port assignments for inter-container communication, workstation proxying, and secure public ingress:

2.1 Complete Port Specification Table

Port Number	Protocol	Config Key	Scope / Binding	Service Name	Technical Function & Description
6000	TCP	APP_PORT	0.0.0.0:6000	raksha_tech	Primary Go backend. Serves API gateway, Web UI, and Browser Guard fleet reporting.
3100	TCP	UI_PORT	127.0.0.1:3100	Vite Dev Server	Local frontend development server (<code>npm run dev</code> in <code>ui/</code>). Inactive in production.
18182	TCP	PROXY_PORT	0.0.0.0:18182	Network Proxy	Optional shared MitM proxy container for centralized office PAC deployments.
18103	TCP	RAKSHA_PROXY_ADD_R	127.0.0.1:18103	Laptop Guard Proxy	Local MitM proxy running on each employee laptop to inspect outbound GenAI traffic.
18195	TCP	PAC_HTTP_PORT	127.0.0.1:18195	Laptop PAC Server	Local HTTP server serving the dynamic Proxy Auto-Configuration (PAC) script to the OS.
5432	TCP	DB_PORT	Internal VPC	PostgreSQL	Production relational database storing configuration, tenants, keys, and audit logs.
11434	TCP	OLLAMA_URL	Docker Bridge	Ollama AI Engine	Local AI inference engine serving offline open-source models via <code>1panel-network</code> .
80	TCP	System Port	0.0.0.0:80	Nginx / Caddy	Inbound public HTTP port for Certbot ACME domain validation and HTTPS redirect.
443	TCP	System Port	0.0.0.0:443	Nginx / Caddy	Public secure HTTPS edge entry point terminating SSL/TLS certificates.

2.2 Perimeter Firewall Security Rules

+-----+				
FIREWALL ACCESS CONTROL MATRIX				
+-----+-----+-----+-----+				
Traffic Policy Port Allowed Source IP Security Policy & Purpose				
+-----+-----+-----+-----+				
PUBLIC INGRESS 443 0.0.0.0/0 (Internet) Main entry point terminating TLS for UI/API PUBLIC INGRESS				
80 0.0.0.0/0 (Internet) ACME HTTP-01 challenge & redirect to HTTPS INTERNAL PRIVATE 6000				
127.0.0.1 / Nginx Host Protect Go backend from direct public access INTERNAL PRIVATE 5432 App				
Containers / VPC Restrict PostgreSQL database to VPC only INTERNAL PRIVATE 11434 App Containers /				
VPC Restrict Ollama LLM to internal backend ENDPOINT LOOPBACK 18103 127.0.0.1 Only Workstation				
proxy bound strictly to local ENDPOINT LOOPBACK 18195 127.0.0.1 Only Workstation PAC server bound				
to local				
+-----+-----+-----+-----+				

3. Docker Installation & Command Handbook

3.1 Download & Install Docker (One-Liner Commands)

- **Ubuntu / Debian Linux (20.04, 22.04, 24.04 LTS):**

```
curl -fsSL https://get.docker.com | sh && sudo systemctl enable --now docker && sudo usermod -aG docker $USER
```

- **CentOS / RHEL 8 & 9 / Rocky Linux:**

```
sudo dnf install -y dnf-plugins-core && sudo dnf config-manager --add-repo
https://download.docker.com/linux/centos/docker-ce.repo && sudo dnf install -y docker-ce
docker-compose-plugin && sudo systemctl enable --now docker
```

- **Microsoft Windows 10 / 11:**

1. Open PowerShell as Administrator and run: **wsl --install**
2. Download Docker Desktop from: <https://docs.docker.com/desktop/install/windows-install/>

3.2 Docker Network Prerequisites (Run Once)

```
docker network create 1panel-network docker network create raksha-network
```

3.3 Daily Docker Operations Command Handbook

```
# --- STARTUP & BUILD OPERATIONS --- # Start Raksha backend in background (detached mode) docker compose up
-d # Start backend AND optional shared network proxy docker compose --profile network-proxy up -d # Rebuild
and launch containers after any code update docker compose up -d --build # Full clean rebuild without Docker
cache docker compose build --no-cache && docker compose up -d # --- MONITORING & LOG OPERATIONS --- # Check
status and health of running containers docker compose ps # Follow real-time live logs of the backend
container docker compose logs -f --tail=100 raksha_tech # Check container CPU %, RAM consumption, and I/O
docker stats raksha_tech # --- IN-CONTAINER DEBUGGING OPERATIONS --- # Open an interactive shell inside the
Raksha container docker exec -it raksha_tech /bin/sh # Test backend healthcheck from inside container docker
exec -it raksha_tech wget -qO- http://127.0.0.1:6000/health # --- STOP, RESTART & MAINTENANCE --- # Restart
backend gracefully (e.g. after editing .env) docker compose restart raksha_tech # Stop and remove containers,
networks, and internal links docker compose down -v # Clean up stopped containers, unused networks, and
dangling images docker system prune -f
```

4. Cryptographic Secrets & Certificates Handbook

All cryptographic keys, authentication tokens, and SSL certificates must be generated using cryptographically secure random sources prior to production launch:

4.1 Secrets for .env File

Environment Secret Key	Standard	Technical Purpose	Exact Generation Command
RAKSHA_GUARD_SECRET	256-bit Random Base64	Fleet Desktop Agent Mutual Auth Token	openssl rand -base64 32
RAKSHA_ENCRYPTION_KEY	256-bit Random Base64	AES-256 DB Credential Encryption at Rest	openssl rand -base64 32
PASSWORD_RESET_SECRET	384-bit Random Base64	User Password Recovery HMAC Token	openssl rand -base64 48
RAKSHA_METRICS_TOKEN	256-bit Random Hex	Prometheus Bearer Token for /metrics	openssl rand -hex 32
CLUSTER_REPLICATE_SECRET	384-bit Random Base64	Multi-Node Cluster State Replication	openssl rand -base64 48

```
# Automated one-liner to generate and inject all secrets directly into .env: sed -i
"s|^RAKSHA_GUARD_SECRET=.*|RAKSHA_GUARD_SECRET=$(openssl rand -base64 32)|" .env sed -i
"s|^RAKSHA_ENCRYPTION_KEY=.*|RAKSHA_ENCRYPTION_KEY=$(openssl rand -base64 32)|" .env sed -i
"s|^PASSWORD_RESET_SECRET=.*|PASSWORD_RESET_SECRET=$(openssl rand -base64 48)|" .env sed -i
"s|^RAKSHA_METRICS_TOKEN=.*|RAKSHA_METRICS_TOKEN=$(openssl rand -hex 32)|" .env sed -i
"s|^CLUSTER_REPLICATE_SECRET=.*|CLUSTER_REPLICATE_SECRET=$(openssl rand -base64 48)|" .env
```

4.2 Public & Private SSL/TLS Certificate Generation

- **Public Let's Encrypt HTTPS (Certbot):**

```
sudo certbot certonly --standalone -d raksha.yourcompany.com --email secops@yourcompany.com --agree-tos
```

- **Private Self-Signed TLS Certificate (OpenSSL 4096-bit RSA with SAN):**

```
openssl req -x509 -nodes -days 1095 -newkey rsa:4096 \ -keyout raksha_server.key -out raksha_server.crt \
-subj "/C=US/ST=State/L=City/O=YesPanchi/OU=IT/CN=raksha.yourcompany.com" \ -addext
"subjectAltName=DNS:raksha.yourcompany.com,DNS:localhost,IP:127.0.0.1"
```

4.3 Browser Guard Root CA Generation & Installation

```
# 1. Generate 4096-bit Root CA Key and 10-year Certificate for MitM proxy openssl genrsa -out raksha-ca.key
4096 openssl req -x509 -new -nodes -key raksha-ca.key -sha256 -days 3650 \ -out raksha-ca.crt -subj
"/CN=Raksha Root CA - Enterprise AI Guard" cat raksha-ca.key raksha-ca.crt > mitmproxy-ca.pem # 2. Deploy
Root CA to Workstation Trust Stores: # Windows (CMD Admin) : certutil -addstore -f "Root" raksha-ca.crt #
macOS (Terminal) : sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain
raksha-ca.crt # Linux (Workstation) : sudo cp raksha-ca.crt /usr/local/share/ca-certificates/ && sudo
update-ca-certificates
```

5. Production Deployment & Verification Matrix

5.1 Quick-Start 4-Step Deployment Checklist

+-----+			
PRODUCTION DEPLOYMENT CHECKLIST			
+-----+			
Step	Action Task	Terminal Command to Execute	
+-----+ 1			
Copy Environment File	cp .env.example .env	2 Generate Secrets Populate RAKSHA_GUARD_SECRET & RAKSHA_ENCRYPTION_KEY in .env	3 Create Docker Network docker network create 1panel-network
Compile & Start App	docker compose up -d --build		4
+-----+			

5.2 Health Checks & Operational Verification

1. Verify Go Backend Healthcheck Endpoint curl -i http://localhost:6000/health # Expected: HTTP/1.1 200 OK {"status":"healthy"} # 2. Verify Container Process Status docker compose ps # Expected: raksha_tech (healthy) Up X minutes 0.0.0.0:6000->6000/tcp # 3. Test PostgreSQL Database Connectivity from Host nc -zv 127.0.0.1 5432 # Expected: Connection to 127.0.0.1 port 5432 [tcp/postgresql] succeeded! # 4. Test Local Ollama Model Server Connectivity curl -i http://127.0.0.1:11434/api/tags # Expected: HTTP/1.1 200 OK with JSON array of installed models
--

5.3 Operational Diagnostics & Resolution Matrix

Symptom / Observed Error	Root Cause Analysis	Corrective CLI Resolution
failed to connect to PostgreSQL on port 5432	Database container down or firewall blocking port	Run docker compose ps to inspect DB; check DB credentials in .env ; verify host port.
network 1panel-network not found	External network was not created prior to launch	Run docker network create 1panel-network , then rerun docker compose up -d .
Browser Guard: 401 Unauthorized on /api/*	Mismatched RAKSHA_GUARD_SECRET between client & backend	Run python apps/browser-guard/scripts/sync_config_from_env.py to synchronize keys.
Browser displays ERR_CERT_AUTHORITY_INVALID	Root CA certificate not installed in client trust store	Install raksha-ca.crt into client OS store (certutil on Windows, security on macOS).
Port 6000 already in use / bind: address already in use	Another process is occupying host port 6000	Run `netstat -ano`
UI displays blank screen or 404 on assets	Web UI static bundle missing from Go binary	Run npm run build:raksha in ui/ , then rebuild with docker compose up -d --build .

Platform Support & Enterprise Maintenance

- **Lead Repository:** **d:\unifai_project**
- **Core Configuration:** **.env** and **configs/config.json**
- **Engineering Parent:** Raksha SecOps & Cloud Infrastructure Group, YesPanchi Group of Companies